

Objektorientierte Programmierung in Java

Rechtliche Bemerkungen:

Das vorliegende Skript
sowie die dazu erstellten
Videos dienen einzig und
allein dazu, Studierende
der TH Ulm zu unterrichten.

Nur dieser Benutzerkreis
darf Skript und Videos zum
eigenen Gebrauch
kopieren und benutzen.

Jegliche darüber hinaus
gehende Weitergabe oder
Verwendung von Skripten
oder Videos oder Teilen
davon ist nicht gestattet.

Insbesondere (aber nicht
ausschliesslich) das weiter-
mailen, kopieren oder
Veröffentlichung in sozialen
Medien oder per Internet ist
nicht gestattet.

Übersetzung: liebe
Studierende meiner
Vorlesung – Sie dürfen die
Dinge hier selber nutzen,
aber bitte nicht irgendwie
weitergeben.

0. Programmieren

Wozu ist es überhaupt sinnvoll, für einen bestimmten Beruf Programmieren zu lernen?
Hier gibt es es natürlich (mindestens) drei Hauptgruppen von Tätigkeiten, solche, die

inhaltlich mit Programmieren zu tun haben (Informatiker, MedizinInformatiker, Simulationsspezialisten etc...)

- hier erklärt sich die Frage von selbst: es gehört dazu

inhaltlich überhaupt nichts mit Programmieren zu tun haben (Journalisten, Theologen, Griechischlehrer etc...)

- hier erklärt sich die Frage auch von selbst: macht meist keinen Sinn. Kann aber Spass machen.

direkt nichts mit Programmieren zu tun haben, in denen aber oftmals Berechnungen oder technische Verfahren verwendet werden (Elektrotechniker, Maschinenbauer, Mathematiker etc...)

- hier kann es Sinn machen, einfache Programmierung zu erlernen, um bestimmte Berechnungen oder Arbeitsabläufe zu automatisieren

Technische Beispiele gibt es zu hauf, mal ein ganz triviales:

- Schraube, Festigkeitsklasse 5.8, 12mm, Feingewinde, welches Anzugsdrehmoment wird benötigt?

Die Formel hierzu ist immer die selbe (Festigkeitsklasse, Durchmesser, Gewindeart), man könnte die einmal programmieren und dann immer wieder als Programm aufrufen.

Betrachten wir das ganze aber durchaus mal etwas "genauer", sprich formaler:

1. Aufgabe der Programmierung

Programmieren in der Informatik bedeutet,

- ein Lösungsverfahren für eine Aufgabe so zu formulieren,
- dass es von einem Computer ausgeführt werden kann

([1]: Informatikhandbuch. Rechenberg, Pommberger)

Das beginnt damit, dass

- zuerst eine grobe Lösungsidee entwickelt wird
- die dann schrittweise verfeinert wird
- bis die Idee so detailliert ist, dass sie in einer „handelsüblichen“ Programmiersprache realisiert werden kann

Beispiel:

Ein Programm in Java, das die Primzahlen bis 1000000 ausdrückt:

```
public class Primzahlen
{
    public static final int MAXIMUM = 1000000;

    public static void main(String[] args)
    {
        for(int i=2; i<MAXIMUM; i++)
        { if(isPrim(i)) System.out.print(""+i+" ");
        }
    }

    private static boolean isPrim(int x)
    {
        for(int test=2; test<x; test++)
        { if((x%test)==0) return false;
        }
        return true;
    }
}
```

In der „guten Alten C-Zeit“ wäre dieses Programm übrigens ggf. wie folgt programmiert worden:

```
void main(){for(int z=2;z<1000000;z++){int t=2;for(;t<z;t++)if(z%t)break;
if(t>=z)printf("%d ",z);}}
```

Programmiersprachen, die heutzutage üblich sind, sind (grob der Wichtigkeit nach geordnet, *siehe zB Tiobe Index*)

- Java (Sun, Oracle, sehr gute Programmierung)
- C++ und C (offen, Unix, gut für Informatiker)
- Python (offen, toll für alle möglichen Techniker)
- C# (Microsoft, eine recht gute "Kopie" von Java)

1.1. Algorithmus

Normalerweise ist es eine gute Idee,

- vor dem „eigentlichen Programmieren“, also dem Erstellen von Programmcode,
- erst einmal eine „logische Idee“ des zu implementierenden Programmes zu bekommen

Eine solche „logische Idee“ wird in der Informatik **Algorithmus** genannt.
Die Erstellung

- eines guten Algorithmus
- für ein vorliegendes Problem
- ist eine **kreative** Tätigkeit, die oftmals nicht leicht zu bewältigen ist

Natürlich kann der Begriff „Algorithmus“ auch etwas formaler definiert werden. Ein Algorithmus ist (wieder [1]):

- eine präzise, d.h. In einer festgelegten Sprache abgefasste
- endliche Beschreibung
- eines schrittweisen Problemlösungsverfahrens
- zur Ermittlung gesuchter Werte
- aus gegebenen Werten
- in dem jeder Schritt aus einer Anzahl ausführbarer

eindeutiger Aktionen

- und einer Angabe über den nächsten Schritt besteht

Einen Algorithmus kann man auch sehen als

- eine Art „Kochrezept“, in dem angegeben wird, wie
- aus den Zutaten
- mittels einzelner, einfacher Schritte
- und gegebenenfalls Wiederholungen von Schritten
- das gewünschte Ergebnis entsteht

In dieser Vorlesung soll nicht tief in das Thema „Mathematik“ eingestiegen werden.
Trotzdem: Schauen wir uns einmal einen Algorithmus zur Berechnung von Primzahlen an.

1.1.1. Beispiel – Ausdrucken von Primzahlen

Zuerst einmal – was ist eine Primzahl. Eine Primzahl ist

- eine natürliche Zahl, deren Teilermenge genau zwei Elemente enthält

Eine solche Definition ist für Zwecke in der Informatik etwas zu „abstrakt“. Nehmen wir eine zweite Definition: Eine natürliche Zahl ist eine Primzahl,

- wenn sie nur durch 1 und durch sich selber geteilt werden kann

Das bedeutet andererseits, (nennen wir das mal PRIMZAHL-FESTSTELLUNG)

- eine Zahl X ist KEINE Primzahl,
- wenn sie durch 2 oder durch 3 oder durch 4 ... oder durch $(X-1)$ geteilt werden kann

Beispiel:

20 ist sicher keine Primzahl (warum?)

23 ist ein guter Kandidat für eine Primzahl

91 ist ein guter Kandidat für eine Primzahl

Testen wir diese Zahlen kurz...

Wir stellen fest:

- erster Verbesserungsvorschlag: wir müssen nicht bis $(X-1)$, sondern nur bis „Wurzel X “ testen...

Wie könnte jetzt ein Algorithmus aussehen, der Primzahlen von 2 bis 1000 ausdrückt?

Mach mit allen Zahlen X von 2 bis 1000 folgendes:
wenn X eine Primzahl ist, dann drucke sie aus

Natürlich müssen wir auch noch den Test, ob X eine Primzahl ist, als Algorithmus formulieren. Nach unserer PRIMZahl-FESTSTELLUNG oben, könnten wir diesen Algorithmus so formulieren:

Algorithmus, ob X eine Primzahl ist:

Mache mit allen Zahlen W von 2 bis (X-1) folgendes:
 wenn X sich durch W ohne Rest teilen lässt,
 dann beende den Algorithmus mit: X ist KEINE Primzahl

Beende den Algorithmus mit: X ist eine Primzahl

Das Programm auf Seite 2

- ist nun genau die Umsetzung dieses Algorithmus
- in der Programmiersprache Java

1.1.2. Beispiel – Zählen von Vokalen in einem Wort

Nehmen wir ein anderes Beispiel: wir wollen in einem Wort zählen, wie viele Vokale darin vorkommen

Beispiel:

Das Wort „informatikhandbuch“ enthält Vokale

Das Wort „schrumpfst“ enthält Vokale

Wie sind Sie bei der Feststellung der Anzahl an Vokalen vorgegangen? Sie haben vielleicht

- mit dem Zählen bei 0 angefangen
- die Buchstaben des Wortes vom ersten bis zum letzten Buchstaben genommen
- und wenn es ein Vokal war, einen dazugezählt

[illegible]

In der Programmiersprache Java lässt sich dieser Algorithmus 1:1 umsetzen:

Algorithmus zum Zählen der Vokale im Wort **x**

```
public int anzahlVokale(String x)
{
```

Setze einen Zähler **vokale** auf 0

```
int vokale = 0;
```

Zähle mit **i** von 0 bis zur Länge von **x**

```
for(int i=0; i<x.length(); i++)
{
```

Mache mit dem jeweils **i**-ten Buchstaben **b** von **x** folgendes:

```
char b = x.charAt(i);
```

wenn **b** ein 'a' oder 'e' oder 'i' oder 'o' .. ist

```
if(b=='a' || b=='e' || b=='i' || b=='o' || b=='u')
```

dann erhöhe **vokale** um eins

```
{ vokale = vokale+1;
```

```
}
```

```
}
```

Ergebnis des Algorithmus ist **vokale**

```
return vokale;
```

```
}
```

Einschub – Installation von Eclipse (eine Java IDE)

Für diese Veranstaltung benötigen Sie ein “Integrated Development Environment” zum Entwickeln in Java. Gut hier sind: IntelliJ (kostet), NetBeans (ist auch ok) und Eclipse (nehmen wir).

Um auf Ihrem Computer zu Hause Java zu entwickeln, sollten Sie Eclipse wie folgt installieren und starten:

00. rufen Sie im Browser die Homepage des Projektes auf: eclipse.org
01. von dort dann zu *Download*
02. was Sie brauchen, ist die “[Eclipse IDE in 64 bit](#)”
03. also laden Sie dort mit *Download* das Installationsprogramm herunter
04. notfalls auf “click here to start the Download” mit rechtem Mausklick speichern
05. Sie sollten jetzt auf Ihrem Rechner eine Datei “[eclipse-inst...exe](#)” als Download haben

06. diese Datei doppelklicken Sie und starten damit den Eclipse-Installer
07. was Sie brauchen (vermute ich stark) ist dann die “[Eclipse IDE for Java Developers](#)”
08. daher: auswählen und *installieren* lassen

09. jetzt -sollte- Eclipse installiert sein und sich starten lassen
10. starten Sie Eclipse per Desktop-Icon *oder* Startmenu *oder* indem Sie in das Suchfeld unten links “[Eclipse](#)” eingeben
11. Wenn Sie nach einem “[Workspace](#)” gefragt werden sollten, lassen Sie den am besten so wie er initial ist.

12. entsprechende “Einführungsprojekte” oder ähnliches klicken Sie bitte einfach weg
13. jetzt haben Sie das “reine” Eclipse vor sich

Einschub – Arbeiten mit Eclipse

- 00. starten Sie Eclipse per Desktop-Icon *oder* Startmenu *oder* indem Sie in das Suchfeld unten links "**Eclipse**" eingeben
- 01. Wenn Sie nach einem "**Workspace**" gefragt werden sollten, lassen Sie den am besten *so wie er initial ist*.
- 02. klicken Sie ggf kommende "**Einführungsprojekte**" und ähnliches weg
- 03. jetzt haben Sie das "reine Eclipse" vor sich

Damit haben Sie Ihr Eclipse gestartet

- 04. für jedes NEUE PROGRAMM sollten Sie ein neues Projekt in Eclipse anlegen:
 ++ *File* → *New* → *Java Projekt*
 ++ geben Sie einen **Projektnamen** ein (zB "Test")

Damit haben Sie ein Projekt erstellt

- 05. jetzt brauchen Sie eine KLASSE, die das Programm darstellt
 ++ *rechterMausclickAufProjekt* → *New* → *Class*
 ++ geben Sie bei "Name" einen **Programmnamen** ein (zB "Hallo")
 ++ **wichtig**: löschen Sie den Eintrag oben im Feld "**Package**"
 ++ bestätigen Sie das ganze mit "OK"

Damit haben Sie im Projekt eine Klasse erstellt

- 06. jetzt müssen Sie Ihre Klasse (Ihr "Programm") programmieren
- 07. dazu öffnen Sie den Pfad zu Ihrer Klasse durch:
 ++ auf **Projektnamen** ("Test") klicken
 ++ auf "**src**" klicken
 ++ auf (**default package**) klicken
 ++ auf **Klassennamen** ("Hallo") doppelclicken

Damit haben Sie den Programmcode Ihrer Klasse vor sich

- 08. jetzt tippen Sie dort hinein den Code aus der Vorlesung

```
class Hallo{public static void main(String[] a){System.out.println("hallo");}}
```

Damit haben Sie Ihre erste Klasse programmiert

- 09. jetzt STARTEN Sie ihr erstes Java-Programm mittels
 ++ *rechterMausclickAuf* **src** → *Run As* → *Java Application*

10. Funktioniert alles? Dann: Eclipse läuft! (Einschub Ende)

2. Java

Java ist eine

- moderne Programmiersprache
- die Objektorientierung bietet (mehr dazu später)
- und aus der Algol-Familie stammt (Oma: Algol, Mutter: C++)

Kurz zur Geschichte von Java: Java wurde hauptsächlich entwickelt von

- **James Gosling**, einem Entwickler bei Sun Microsystems,
- der C++ (die damals favorisierte Programmiersprache)
- als zu unhandlich und unnötig kompliziert empfand
- und deswegen für Sun eine neue, eigene Programmiersprache entwickelte.
- Java hiess zuerst "Oak" (Eiche), u.a. wegen eines entsprechenden Baums vor Goslings Fenster

Java vereint folgende Punkte sehr gut miteinander:

- kurzen, handlichen Code auf der Basis von C++ Syntax
- gegenüber C++ deutlich verbesserte Datenkonzepte (References, Garbage Collection)
- einer klaren objektorientierten Struktur
- hohen Geschwindigkeit
- sehr hohe Portierbarkeit zwischen verschiedenen Betriebssystemen (Unix, Linux, Windows, MacOS, Android, RaspberryPi, ...)

Es gibt nur sehr wenige mögliche ggf. kritische Punkte an Java:

- am wichtigsten ist sicher, das Java (im Gegensatz zu C++) keinen direkten Zugriff auf Hardware erlaubt
- Microsoft ist definitiv KEIN Fan von Java

Wir wollen in diesem Semester zusammen einen Einstieg in die objektorientierte Programmierung bekommen. Mit **Java**,

- aber Java soll nicht im Zentrum stehen, sondern *objektorientierte Konzepte*

Daher ist es hier und da möglich, dass in diesem Skript

- Details zu java-spezifischen Konstrukten, die es in manchen anderen Objektorientierten Sprachen nicht gibt
- bewusst weggelassen, verkürzt oder vereinfacht werden

2.1. Grundstruktur von Java-Programmen

Im folgenden soll das "einfachste" mögliche Java-Programm kurz dargestellt und erklärt werden:

```
public class Hallo
{
    public static void main(String[] args)
    {
        // hier wird ausgedruckt:
        System.out.println("cup of coffee");
    }
}
```

Dieses Programm hat folgende Struktur:

1

public class Hallo

In dieser Zeile wird eine neue Klasse namens **Hallo** definiert.

Jedes Java-Programm beginnt damit, eine oder mehrere neue Klasse zu definieren

(die natürlich nicht unbedingt Coffee heißen muss...)

Klasse

Ein Programm in Java besteht aus einer oder mehreren Klassen.

Klassen stellen somit die "Grundeinheiten" dar, aus denen ein Java-Programm aufgebaut ist.

Klassen- und Dateiname

Als Name für eine Klasse kann in Java eine beliebige Kette aus Buchstaben und Zahlen verwendet werden. Die Zeichenkette muss allerdings mit einem GROSS-Buchstaben beginnen.

Gross- und Kleinschreibung werden in Java strikt unterschieden!

Wenn die Klasse einen Namen XYZ besitzt, muss der Name der Datei, in der sich die Klasse befindet, XYZ.java lauten!

2 **{ ... }** ("erste bis letzte Klammer")

Die Definition jeder Klasse in Java **beginnt mit einer öffnenden geschweiften Klammer und endet mit einer schliessenden geschweiften Klammer**

Innerhalb dieser Klammern **befinden sich die Variablen und Funktionen der Klasse**

3 `public static void main(...)`
`{ ... }`

"main" ist die **Hauptfunktion jeder Klasse.**

Beim "Start" einer Klasse als "Hauptprogramm" durch den Java-Interpreter **wird automatisch diese main-Funktion aufgerufen.**

Die geschweiften Klammern und das darin enthaltene Programmstück **bilden den Block, der ausgeführt wird, wenn die Klasse gestartet wird.**

Block

Ein **Block** ist in Java **ein Programmstück, das mit einer öffnenden geschweiften Klammer beginnt und mit einer schliessenden geschweiften Klammer endet.**

In einem Block steht **eine Reihe von Anweisungen (siehe später...).**

Der **Ausführungsteil** jeder Funktion und Prozedur in Java besteht aus genau einem Block.

4

public

Eine Funktion, die als public markiert wird, ist für "jeder-mann" bekannt und kann von jedem genutzt werden.

5

static

Eine Funktion, die als static markiert wird, gilt "für die ganze Klasse und nicht für einzelne Objekte" (nähere Erklärung siehe später...)

6

void

Eine Funktion, die als void markiert wird, besitzt keinen "Rückgabewert".

Damit ist sie eine Prozedur und keine "eigentliche Funktion im engeren Sinne".

Die main-Funktion ist standardmässig eine solche void-Funktion

7

...(String[] args)

Bei der Definition einer Funktion befinden sich hinter dem

Funktionsnamen **in Klammern die Parameter, die diese Funktion besitzt.**

Die main-Funktion besitzt standardmässig als Parameter

- ein Array (durch die Klammern **[]** charakterisiert)
- von Zeichenketten (durch das Schlüsselwort **String** charakterisiert).

String

String ist die in Java **standardmässig eingebaute Klasse zur Repräsentation von Zeichenketten.**

String-Konstanten **sind beliebige Folgen von Buchstaben und Sonderzeichen, die zwischen zwei Anführungszeichen stehen.**

Beispiel für korrekte Zeichenketten-Konstanten:

```
"hallo" , "Kaffee" , "34duda45&(!$$45",
"MerkurVenusErdeMars" ,
"vom Eise befreit sind Strom und Bäche" , ""
```

9

// ...

Die beiden hintereinanderstehenden Schrägstriche beginnen einen Zeilenkommentar.

Kommentar - No 1

Kommentare werden in Java dadurch erzeugt, dass zwei direkt hintereinander stehende Schrägstriche geschrieben werden.

Der Text hinter den Schrägstrichen bis zum Zeilenende wird als Kommentartext aufgefasst und nicht als Programm übersetzt.

Diese Art von Kommentar wird Zeilenkommentar genannt.

Anweisung

Eine Anweisung ist in Java ein einzelner "Sprachbefehl" wie beispielsweise ein(e)

- Variablendeklaration
- Variablenzuweisung
- Aufrufe einer Funktion
- Kontrollstrukturen (wie z.B. if-...-else-...)

Jede Anweisung muss in Java grundsätzlich mit einem Semikolon beendet werden.

Hinter einen **Block** wird allerdings kein Semikolon gesetzt

Beispiele für Anweisungen

```
String s;      // s ist eine Zeichenkette
s = "hallo";   // s wird der Wert "hallo" zugewiesen
int i = 11;    // i ist eine Zahl mit Startwert 11
i = i*3;       // Änderung des Wertes von i
if(i<50) { System.out.println("klein"); }
else          { System.out.println("gross"); }
```

2.2 Eingebaute primitive Datentypen

Programme dienen bekanntermassen dazu, Daten zu verarbeiten.

Diese Daten werden in Programmen in sogenannten "Variablen" gespeichert.

Eine Variable in Java

- ist eine Art "Platzhalter" für einen bestimmten Wert,
- der Wert kann sich im Laufe des Programmes auch ändern

jede Variable in Java

- hat einen bestimmten, fest definierten Namen
- und einen bestimmten, fest definierten Datentyp (oder auch kurz: Typ)

Der Typ einer Variablen sagt aus,

- welche Art von Daten die Variable speichern kann,
- wie beispielsweise Zahlen, Texte, Farben oder ähnliches

Einen Sonderfall nehmen die 8 sogenannten **primitiven Datentypen** in Java ein: sie sind

- die 8 grundlegendsten Datentypen und
- wegen ihrer häufigen Verwendung
- etwas unterschiedlich zu den restlichen Hunderten von Datentypen in Java implementiert.

Die primitiven Datentypen sollen im folgenden kurz vorgestellt werden.

boolean

Der primitive Datentyp boolean dient in Java zur Repräsentation von booleschen Werten (Wahrheitswerten).

Für boolesche Variablen existieren die mögliche Werte `true` (wahr) und `false` (falsch)

Beispiel für die Definition von boolean Variablen:

```
boolean b;  
boolean wert = false;  
boolean xy   = true;
```

char

Der primitive Datentyp char dient in Java zur Repräsentation von einzelnen Zeichen (engl.: characters).

Character-Konstanten werden in einfache Anführungszeichen eingeschlossen (zB: `'r'`)

Spezielle Character-Konstanten sind

```
'\n' (neue Zeile)  
'\t' (Tabulator)  
' ' (Leerzeichen)
```

Beispiel für die Definition von char Variablen:

```
char c;  
char d = 'x';  
char buchstabe = 'y';
```

int und **long**

Die primitiven Datentypen **int** und **long** dienen in Java zur Repräsentation von ganzen Zahlen.

Dabei besitzt **int** eine Länge von 4 Byte und **long** eine Länge von 8 Byte.

Beispiel für die Definition von ganzzahligen Variablen:

```
int i;  
int j = 55678387;  
long m;  
long n = 290384650745783459L;
```

double und **float**

Die primitiven Datentypen **double** und **float** dienen in Java zur Repräsentation von Gleitkommazahlen.

Dabei sind **double**-Zahlen deutlich genauer als **float**-Zahlen.

Wenn keine anderen Anforderungen gestellt sind, sollte grundsätzlich **double** statt **float** verwendet werden.

Beispiel für die Definition von Gleitkomma Variablen:

```
float f;  
double d;  
double pi = 3.14159265358979;  
double milliarde = 1e+9;  
double schulden = -7e+11; // 700 Milliarden negativ
```

void

Das Schlüsselwort **void** drückt in Java die "Abwesenheit" von Typinformation aus ("kein Typ").

Dies kann beispielsweise dazu dienen, eine Funktion als "ohne Rückgabety" (=Prozedur) zu kennzeichnen.

Beispiel

Das folgende Programm benutzt die vorgestellten Datentypen:

```
public class Datentypen
{
    public static void main(String[] args)
    {
        boolean b = false;
        boolean c = true;
        System.out.println("boolean:"+b+" "+c);

        long h = 123;
        long i = 456;
        long j = h+i;
        System.out.println("long:"+h+" "+i+" "+j);

        double d = 123;
        double e = d*d*d*d;
        double f = e*e*e*e;
        System.out.println("double:"+d+" "+e+" "+f);
    }
}
```

Das Programm besitzt folgende Ausgabe:

```
boolean: false true
long: 123 456 579
double: 123 2250000 1.8e+25
```

2.3 Variablen und Deklarationen

Bevor ein Name (Bezeichner) in einem Java Programm verwendet werden kann, muß er deklariert werden. Das heißt,

- sein Typ muß spezifiziert werden,
- um Java über die Art des Elementes zu informieren

Beispiele für Deklarationen:

```
int wert;
final double pi = 3.141592653;
String vorlesung = "Programmieren 2";
double quadrat(double x) { return x*x; }
```

Eine Deklaration kann gleichzeitig auch eine Initialisierung enthalten. Dabei wird

dem deklarierten Element gleichzeitig ein Wert zugewiesen

Für Funktionen und Konstanten ist dieser Wert konstant. Für nichtkonstante Datentypen kann der Initialwert später geändert werden.

Struktur von Deklarationen

Eine Deklaration hat folgende formale Form:

ggf. Modifika- toren	BASISTYP	NAME	ggf. Deklarations operator	ggf. Initialisierung
----------------------------	----------	------	----------------------------------	-------------------------

In Java werden die folgenden Deklarator-Operatoren verwendet:

[]	Array
()	Funktion/Methode

Weiterhin kennt Java unter anderem die folgenden Modifikatoren:

private protected public	regeln die Zugriffsrechte auf die Variable bzw. Methode (später...)
---	---

static	Variable bzw. Methode gehört zur Klasse und nicht zum Objekt
---------------	---

final	- Variable ist eine Konstante - bzw. Methode (später...) ist "unveränderlich"
--------------	---

Beispiele für Deklarationen wären dementsprechend:

	int	i	= 5
	String	name	= "Burns"
public static	void	main()	
	long	lo	

Namen

Namen (z.B. für eigene Klassen, Methoden und Variablen) bestehen in Java:

- aus einer Folge von Buchstaben und Ziffern
- das erste Zeichen muß ein Buchstabe sein
- der Unterstrich wird dabei als Buchstabe betrachtet (sollte aber nie am Namensanfang stehen!)

Beispiele für Namen:

i
nachname
arrayAllerKunden
vorname2
berechneDurchschnitt
median

Gültigkeitsbereich

Eine Variable dient dazu, mit dem Variablennamen einen gewissen Wert zu verbinden. Dabei stellt sich zu jeder Variablen die interessante Frage, WO überall im Java-Programm diese Variable "bekannt" ist.

Die Gesamtheit aller "Stellen", an denen eine Variable bekannt ist, **wird auch der Gültigkeitsbereich dieser Variablen genannt.**

Für den Gültigkeitsbereich einer Variablen spielt der Begriff des Blocks eine grosse Rolle:

Block

Ein Block ist ein Programmstück in Java, das { durch geschweifte Klammern eingeschlossen } ist.

Ein Block wird in Java "wie eine einzelne Anweisung" angesehen.

Beispiel für einen Block

```
{   int x = 12;
    int y = x*x;
    System.out.println(y);
}
```

Gültigkeitsbereich

Der Gültigkeitsbereich einer Variablen

- beginnt mit der Stelle, an der die Variable deklariert ist und
- endet mit dem Ende des Blocks, in dem die Variable deklariert ist.

Das bedeutet unter anderem, dass eine Variable, die in einem Block definiert ist, auch in den "Unterblöcken" dieses Blockes gültig ist.

Innerhalb des Gültigkeitsbereiches einer Variablen darf keine andere Variable mit demselben Namen deklariert werden.

Beispiel für Gültigkeitsbereiche (mit Fehlern!):

```
{
    int a = 10;
    System.out.println(" "+a+" "+b+" "+c);
    {
        int b = 22;
        System.out.println(" "+a + " " + b + " " + c);
        int c = 24;
    }
    {
        int b = 30;
        b = b + 5;
        a = a + 5;
        System.out.println(" "+a+" "+b+" "+c);
    }
    System.out.println(" "+a+" "+b+" "+c);
} // zur ersten Klammer
```

Programmierungs-Styleguide

1. Deklarieren Sie nur einen Namen pro Deklaration
2. Verwenden Sie für "längerfristige" Variablen (= Variablen mit "grossem" Gültigkeitsbereich) **aussagekräftige** Bezeichnungen.
3. Solche Namen sollen die **Bedeutung des Elementes in der realen Welt** widerspiegeln!!
4. Halten Sie ein konsistentes Namensschema ein:

	Schreibweise	Beispiele
selbstdefinierte Klassen	fangen groß an; Teilwörter jeweils mit großem Anfang	Vektor3D KundenDaten ProduktionsDaten Zeichenkette
Methoden	fangen klein an; Teilwörter jeweils mit großem Anfang; erstes Wort Verb	berechneKundenDaten addiereAlleVektoren loescheAktuellenKunden
Variablen	am Anfang Kleinbuchstabe; Teilwörter mit Großbuchstaben getrennt	x wert werteMinBisMax alteListeMitPersonen

Operatoren

Operatoren sind "Funktionen" in Java, die

- Ohne runde Klammern vor, zwischen oder hinter zwei Elementen stehen

Operatoren sind aus der Mathematik bekannt, beispielsweise

- der + Operator
- der / Operator
- und ähnliche "grundlegende" Funktionen

Im folgenden eine Tabelle mit den wichtigsten Operatoren in Java:

Operator (en)	Bedeutung	Beispiele
.	Punkt-Operator	<code>s.length()</code>
[]	Index-Operator	<code>feld[12]</code>
()	Funktions-Klammern	<code>sin(1.67)</code>
++	Increment-Operator (um 1 erhöhen)	<code>x++</code>
--	Decrement-Operator (um 1 erniedrigen)	<code>y--</code>
!	logisches Nicht	<code>! (x<5)</code>
new	Erzeugen eines neuen Elementes	<code>x = new String();</code>
* / %	Multiplikation, Division, Modulo	<code>(x*3)/7</code>
+ -	Addition, Subtraktion	
< >	Größen-Vergleiche	<code>if (x < 3) ...</code>
<= >=		<code>if (y >= x) ...</code>
== !=	Gleichheit, Ungleichheit	<code>if (x == 3) ...</code>
&&	logisches Und	<code>if ((x>0) && (x<100)) ...</code>
	logisches Oder	<code>if ((y<0) (y>100)) ...</code>
=	Zuweisung	<code>x = 55</code>
+= /=	andere Zuweisungen	<code>x += 5</code>

Beispiele für Elemente in Java mit Verwendung von Operatoren:

3+5 :

3*(8+5) :

100-7*8 :

x>10 && x<100 :

y == 1 :

a==0 || b==0 :

100%9 :

x += 10 :

a-- :

c++ :

3 Kontrollstrukturen

Wird ein Computerprogramm gestartet, so läuft es "normalerweise" in der Reihenfolge ab, in der die Programmbefehle niedergeschrieben sind.

Kontrollstrukturen sind Strukturen in Computerprogrammen, die es erlauben, **den Programmfluss von dieser sequentiellen Abarbeitungsreihenfolge abweichen zu lassen.**

Nur durch Kontrollstrukturen ist es also möglich, dass ein Programm nicht jedesmal "gleich abläuft", sondern auf

- verschiedene Daten
- verschieden Benutzereingaben
- verschiedene Zustände der "Umwelt"

unterschiedlich reagieren kann.

Die sechs in Java eingebauten grundlegenden Kontrollstrukturen sind

if...
if...else...
while
do...while
for
switch

Diese Konstrukte sollen in den folgenden Abschnitten vorgestellt werden.

Auswahl: *if*, *if...else* und *switch*

Das `if`-Konstrukt dürfte die wohl meistgebrauchteste Kontrollstruktur in den meisten Programmiersprachen sein.

if ...

Die Kontrollstruktur

`if (boolscherAusdruck) Anweisung`

dient dazu,

die Anweisung genau dann auszuführen, wenn der boolscheAusdruck wahr ist.

Beispiele:

```
int x=10;
```

```
if(x < 20)    x = x*3;    // x wird 30
```

```
if(x < 20)    x = x*4;    // x bleibt 30
```

```
if(x < 100)           // Block statt einzelner
{                     // Anweisung
    x = x+5;
    x = x*10;          // x wird 350
}
```

```
if(x < 200)
{    x = x+100;        // x bleibt 350
}
```

Das `if...else...` Konstrukt erweitert das `if`-Konstrukt um den sog. "else-Zweig". Der else-

Zweig einer Auswahlanweisung wird genau dann ausgeführt, *wenn der Wahrheitswert hinter dem if nicht zutrifft.*

if ... else ...

Die Kontrollstruktur

```
if (BooleanAusdruck) Anweisung1 else Anweisung2
```

dient dazu,

die Anweisung1 genau dann auszuführen, wenn der BooleanAusdruck wahr ist.

Ansonsten (wenn der BooleanAusdruck falsch ist), wird Anweisung2 ausgeführt.

Beispiele:

```
int x=10;
```

```
if(x < 20)    x = x*3;
else         x = x*4;    // x wird 30
```

```
if(x < 20)    x = x*4;
else         x = x*5;    // x wird 150
```

```
if(x < 100)           // Blöcke statt einzelner
{                     // Anweisungen
    x = x+5;
    x = x*10;
}
else
{
    x = x + 11;
    x = x * 2;
}                     // x wird 322
```

Einschub: Klasse “Leser” zum Einlesen von Zahlen und Strings

Die folgende Klasse ist etwas “tricky” programmiert, um kompakt zu bleiben und noch auf diese Seite zu passen...

```
import java.io.*;

public class Leser
{
    private BufferedReader reader
        = new BufferedReader(new InputStreamReader(System.in));

    public String readString()
    {
        System.out.print("-> ");
        System.out.flush();
        try { return reader.readLine(); }
        catch(Exception ex) { return null; }
    }

    public long readLong()
    {
        while(true)
        { try { return (Long.parseLong(readString())); }
          catch(Exception ex) {}
        }
    }

    public double readDouble()
    {
        while(true)
        { try { return (Double.parseDouble(readString())); }
          catch(Exception ex) {}
        }
    }

    public int readInt()
    { return (int)readLong();
    }
}
```

Verwendet würde der "Leser" in einer beliebigen Klasse wie folgt:

```
public class IrgendeineKlasse
{
    public static void main(String[] args)
    {
        Leser leser = new Leser();

        double x = leser.readDouble();
        System.out.println("Sinus davon ist: "+Math.sin(x));

        int    y = leser.readInt();
        System.out.println("Quadrat davon ist: "+(y*y));

        String s = leser.readString();
        System.out.println("ich habe gelesen: "+s);
    }
}
```

Möglich sind die Methoden

```
leser.readInt()
leser.readLong()
leser.readDouble()
leser.readString()
```

Verschachtelte if...else... - Konstrukte

Oft liegt bei einer Unterscheidung eine Anzahl von mehr als zwei Fällen vor. Eine solche Unterscheidung **kann durch eine *einzig*e if...else... Anweisung nicht getroffen werden, da diese Anweisung ja nur zwei verschiedene Fälle unterscheiden kann.**

Dennoch können auch mehrfache Unterscheidungen durch if...else... Anweisungen bearbeitet werden, **indem mehrere if...else... Anweisungen ineinander verschachtelt werden.**

Beispiel

Es sollen zwei Zahlen a und b miteinander verglichen werden. Wenn

- a kleiner als b ist, wird "kleiner" ausgedruckt
- a größer als b ist, wird "größer" ausgedruckt
- a gleich b ist, wird "gleich" ausgedruckt

Die Programmlogik dazu sieht wie folgt aus:

Der Java-Code dazu sieht wie folgt aus:

```
// zum Beispiel...:
int a = 5;
int b = 11;

// verschachtelte if...else...'s:
if(a==b)
{   System.out.println("a ist gleich b");
}
else
{
    if(a < b)
    {   System.out.println("a ist kleiner b");
    }
    else
    {   System.out.println("a ist grösser b");
    }
}
```

Schleifen

In der Realität (und auch in der Programmierung) ist es jedoch oft notwendig, daß

- bestimmte Programmteile
- in Abhängigkeit von einem logischen Ausdruck
- durchlaufen werden,
- solange dieser Ausdruck wahr ist.

Für diese Aufgabe werden in Java die Schleifenkonstrukte `while...` und `do...while` verwendet.

while

Die Kontrollstruktur

while (BooleanAusdruck) Anweisung

realisiert eine **abweisende Schleife** und dient dazu,

die Anweisung **solange** auszuführen, wie der BooleanAusdruck wahr ist

Wenn der BooleanAusdruck nicht mehr wahr ist, **wird mit der Programmausführung hinter dem while-Konstrukt fortgesetzt.**

Beispiel (alle durch 4 teilbaren Zahlen zwischen 0 und 50)

```
int i = 0;
```

```
while(i<=50)
```

```
{
```

```
    if( (i%4) == 0)
```

```
    {
```

```
        System.out.println("durch 4 teilbar: "+i);
```

```
    }
```

```
    i = i+1;    // bedeutet das gleiche wie:    i++;
```

```
}
```

do....while

Die Kontrollstruktur

do Anweisung **while** (BooleanAusdruck)

realisiert eine nichtabweisende Schleife und dient dazu,

die Anweisung **solange** auszuführen, wie der BooleanAusdruck wahr ist.

Dabei wird die Anweisung beim **ersten Mal** jedoch ausgeführt, **ohne** den BooleanAusdruck zu prüfen.

Wenn der BooleanAusdruck nicht mehr wahr ist, **wird mit der Programmausführung** hinter dem do...while-Konstrukt fortgesetzt.

Bemerkung:

Es sollte grundsätzlich **immer besser die while Schleife verwendet werden.**

Die **do-while** Schleife sollte nur verwendet werden, wenn dies unbedingt notwendig ist.

Übungsaufgaben

Schleifen

Schreiben Sie ein Programm, das wie folgt anfängt:

```
int sternchen = 20;
```

a

Das Programm soll dann "sternchen"-mal das Zeichen * auf dem Bildschirm (in einer Zeile) ausgeben

Schleifen

Schreiben Sie ein Programm, das wie folgt anfängt:

```
int zahl = 420;
```

b

Das Programm soll dann alle "Teiler" der zahl ausgeben.

Eine Zahl x ist genau dann eine "Teiler" einer anderen Zahl y, wenn x y ohne Rest teilt.

Beispiel:

für zahl=12 wären die Teiler, die ausgedruckt werden:

1 2 3 4 6 12

Weitere Kontrollstrukturen

break

Die **Anweisung**

break

dient dazu, eine der Kontrollstrukturen while, do...while, for, oder switch sofort zu verlassen.

Es wird dann mit der Programmausführung direkt hinter der Kontrollstruktur fortgesetzt.

Beispiel:

Es soll nach dem eben beschriebenen Verfahren der kleinste "echte" Teiler (größer als 1) einer Zahl `zahl` gefunden werden. Das Java-Programm dazu ist:

```
long zahl = 143;

long i = 2;
while(i <= zahl)
{
    if( (zahl%i) == 0 )
    {
        System.out.println(""+i+" teilt "+zahl);
        break;
    }
    i = i+1;
}
```

Verständnisfrage:

Was passiert, wenn das break im obigen Programm weggelassen wird?

for

Schleifen werden häufig verwendet, um eine Variable zu erhöhen (oder zu erniedrigen) und einen Anweisungsblock in Abhängigkeit von dieser Variablen eine bestimmte Anzahl von Malen zu wiederholen.

Beispiel:

Um aus einer Zahl p den Wert "z hoch p" zu berechnen, eignet sich folgendes Programmstück:

```
long p = 6;
long z = 3;

long result = 1;
long i      = 0;
while(i < p)
{
    result = result * z;
    i      = i+1;
}

System.out.println(result);
```

Verständnisfrage:

was tut dieses Programmstück?

Anhand dieses Beispiels und vielen anderen Beispielen lässt sich erkennen, dass die Struktur "typischer" Schleifen häufig wie folgt aussieht (nur skizziert, kein "echtes" Java):

Initialisierung einer Zähl-Variable

WHILE(Bedingung mit Zähl-Variable)

```
{  
    "eigentliche" Befehle  
    Veränderung der Zähl-Variable // oft vergessen  
}
```

Um diese "typischen" Zählschleifen kürzer schreiben zu können, wurde die `for` Kontrollstruktur entwickelt:

for

Die Kontrollstruktur

```
for(Initialisierung ; Bedingung ; ÄnderBefehl)  
{ Block  
}
```

ist eine "Abkürzung" für "typische" Zählchleifen, die wie folgt abläuft:

- Initialisierung ist ist eine einzige Zuweisung
oder mehrere Anweisungen, durch Kommas getrennt
- ÄnderBefehl ist eine einzige Variablenänderung
oder mehrere Anweisungen, durch Kommas getrennt
- zuerst wird Initialisierung genau einmal
ausgeführt.
- solange die Bedingung gilt,
- wird der Block (hier kann statt des Blockes
auch eine einzelne Anweisung stehen)
ausgeführt.
- dann wird der ÄnderBefehl ausgeführt.

Beispiel:

Das obige Programmstück (um aus einer Zahl p den Wert "z hoch p" zu berechnen) lässt sich wie folgt kürzer formulieren:

```
long p      = 6;
long z      = 3;
long result = 1;

for(long i=0; i<p; i++)
{
    result = result*z;
}

System.out.println(result);
```

for und break

Die **break** Anweisung lässt sich auch benutzen, um die aktuelle **for**-Schleife zu verlassen.

Beispiel: numerisches Integrieren

Im folgenden Beispiel soll eine Funktion zwischen 2 Grenzen numerisch integriert werden. Dabei werden die Grenzen durch ein "Leser"-Objekt eingelesen.

```
// Grenzen des Integrals einlesen
Leser leser = new Leser();

System.out.println("bitte Anfangs- und Endwert
                    eingeben");
double anfang = leser.readDouble();
double ende   = leser.readDouble();

// Schrittweite zum numerischen integrieren
double schrittweite = 0.001;

double result = 0.0;

// x ist der jeweilige Wert, wo f(x) berechnet wird
for(double x=anfang; x<ende; x=x+schrittweite)
{
    // zu integrierende Funktion : f(x) berechnen...
    double funktionswert = Math.pow(x,2);

    // Flaeche des Streifens addieren
    double flaeche = schrittweite*funktionswert;
    result      = result + flaeche;
}

System.out.println("ergibt integriert: "+result);
```

Übungsaufgaben

for Schleife

Schreiben Sie beiden letzten die Übungsaufgaben A und B für Schleifen so um, dass sie mit **for**-Schleifen statt mit **while**-Schleifen implementiert werden

a

for Schleife

Implementieren Sie ein Programm, das Temperatur-Umrechnungstabellen erstellen kann!

Die Temperatur-Umrechnung soll von Grad Fahrenheit (amerikanisches System) nach Grad Celsius (internationales System) erfolgen.

Ein Celsiuswert C ergibt sich dabei aus einem Fahrenheitwert F nach der Formel

$$C = (F - 32.0) * 5.0 / 9.0$$

Definieren Sie den Anfang und das Ende der Tabelle durch zwei double-Variablen mit den Namen `anfang` und `ende`!

for Schleife

Implementieren Sie ein Programm "Fakultät", das Ihnen zu einer Zahl `zahl` die Fakultätsfunktion berechnet und auf dem Schirm ausgibt.

Fakultätsfunktion (in der Statistik benötigt):

Wenn N eine Zahl ist, dann ist die Fakultät von N = $1 * 2 * 3 * \dots * N$

Beispiel: die Fakultät von 6 ist $1 * 2 * 3 * 4 * 5 * 6$ gleich 720

c

Verschachtelte Schleifen

Schleifenkonstrukte lassen **sich auch ineinander verschachteln**.

Wenn wir beispielsweise ein Rechteck mit **zeilen** Zeilen und **spalten** Spalten ausdrucken lassen wollen, bietet sich folgender Algorithmus an:

lies zeilen und spalten ein
 zähle mit y von 1 bis zeilen:
 zähle mit x von 1 bis spalten:
 drucke ein "o" (OHNE neue Zeile)
 drucke eine neue Zeile

Der entsprechende Java-Code sieht dann so aus:

```

Leser leser = new Leser();
System.out.println("bitte Zeilen und Spalten eingeben");
int zeilen  = leser.readInt();
int spalten = leser.readInt();

for(int y=1; y<=zeilen; y++)
{
    for(int x=1; x<=spalten; x++)
    {
        System.out.print("o");
    }
    System.out.println();
}
  
```

Übungsaufgaben

Schleifen 2

a

Schreiben Sie ein Programm, dass

- eine Zahl "kante" einliest
- und dann ein Dreieck der Höhe und Breite "kante" zeichnet.

Beispiel bei "kante" gleich 4:

```
*
* *
* * *
* * * *
```

Schleifen 2

b

Schreiben Sie ein Programm, das das "kleine Einmaleins" auf den Bildschirm zeichnet, also:

```
1 2 3 4 5 6 7 8 9
2 4 6 8 10 12 14 16 18
3 6 9 12 15 18 21 24 27
....
```

Schleifen 2 (schwierig!!)

c

Schreiben Sie ein Programm, das

- eine Zahl "hoehe" einliest
 - und dann einen Weihnachtsbaum dieser Höhe zeichnet
- Beispiel bei "höhe gleich 4:

```
      *
    * * *
  * * * * *
* * * * * * *
  ||
```

Arrays

Eine Variable dient dazu, in einem Programm **einen bestimmten Wert zu speichern.**

Es passiert allerdings manchmal, dass

- nicht nur ein Wert,
- sondern mehrere Werte eines gleichen Typs
- oder eine "Reihe" von Werten des gleichen Typs

gespeichert werden soll. Dies ist beispielsweise der Fall bei

- einer Reihe von Messwerten
- einem Text mit vielen Zeilen
- oder mathematischen Konstrukten wie Vektoren oder Matrizen

In einem solchen Fall werden keine "einfachen" Variablen verwendet, **sondern sogenannte Arrays (oder: Felder).**

Array

Ein Array (oder: Feld)

- ist eine Datenstruktur aus mehreren Elementen
- vom gleichen Typ
- die automatisch "durchnummeriert" werden

Auf ein bestimmtes Element eines Arrays

- kann über eine ganze Zahl zugegriffen werden
- diese Zahl wird "Index" genannt
- und kann zwischen 0 und der Länge des Arrays minus 1 liegen

Bei Deklaration und Verwendung von Arrays

- werden die eckigen Klammern verwendet

Die Schreibweise bei der Deklaration des Arrays lautet

Datentyp[] arrayName = new Datentyp[arrayLänge];

Die Schreibweise beim Zugriff auf das Element Nummer n des Arrays lautet

- arrayName[n]
- wobei n (der Index) zwischen 0 und der arrayLänge-1 liegen darf

Beispiel:

Ein Array von 10 double-Zahlen würde wie folgt deklariert werden:

`double[] werte = new double[10];`

Bildlich kann man sich ein solches Array wie folgt vorstellen:

Beispiel:

Es sollen 5 Messwerte eingelesen und danach

- die Messwerte
- der Durchschnitt der Messwerte ausgegeben werden

Der Algorithmus dazu sieht wie folgt aus:

definiere ein Array messwerte

// einlesen:

zähle mit n von 0 bis 4:

lies den messwerte nummer n ein

// auswerten:

definiere summe als 0.0

zähle mit n von 0 bis 4:

gib den messwerte nummer n aus

addiere den messwerte nummer n zu summe

gib den Durchschnitt (summe/5) aus

Der Java-Code dazu sieht so aus:

```
Leser leser = new Leser();

double[] messwerte = new double[5];

// Messwerte einlesen
for(int n=0; n<5; n++)
{
    System.out.println("bitte Messwert "+n+" eingeben:");
    messwerte[n] = leser.readDouble();
}

// Messwerte ausgeben und Summe berechnen
double summe = 0.0;
for(int k=0; k<5; k++)
{
    System.out.println("Wert Nr. "+k+" : "+messwerte[k]);
    summe = summe+messwerte[k];
}

// Durchschnitt ausgeben
System.out.println("Durchschnitt: "+(summe/5.0));
```

Erweiterung des Programmes

Von dem Programm soll weiterhin gefordert werden, dass es

- nach der Ausgabe des Durchschnitts
- auch noch das *Maximum* der Messwerte sucht
- und ausdruckt

Der “zusätzliche” Algorithmus-Teil dazu sieht wie folgt aus:

Definiere maximum als messwerte[0]

zähle mit n von 1 bis 4

wenn messwerte[n] grösser ist als maximum

dann ist das maximum jetzt messwerte[n]

Gib maximum aus

Der Java-Code dazu ist:

```
double maximum = messwerte[0];
for(int n=1; n<5; n++)
{
    if(messwerte[n] > maximum)
    { maximum = messwerte[n];
    }
}
```

```
System.out.println("Maximum ist: "+maximum);
```

```
public class Messwerte
{
    public static void main(String[] args)
    {
        Leser leser = new Leser();

        double[] messwert = new double[5];

        for(int n=0; n<5; n++)
        {
            System.out.println("bitte Messwert "+n+" eingeben:");
            messwert[n] = leser.readDouble();
        }

        double summe = 0.0;
        for(int n=0; n<5; n++)
        {
            System.out.println("Wert Nr. "+n+" : "+messwert[n]);
            summe = summe+messwert[n];
        }

        System.out.println("Durchschnitt: "+(summe/5));

        double maximum = messwert[0];
        for(int n=1; n<5; n++)
        {
            if(messwert[n]>maximum)
            {
                maximum = messwert[n];
            }
        }

        System.out.println("Maximum ist: "+maximum);
    }
}
```

Mehrdimensionale Arrays

In der Mathematik und in der Programmierung existieren manchmal nicht nur eindimensionale Arrays, sondern auch

- zwei- oder mehrdimensionale Array,
- beispielsweise für Matrizen (2-dimensional)
- oder in Spielen wie Schach oder Go (2-dimensional)
- oder bei Messwerten aus der Computertomografie oder der Finite-Element-Methode (3-dimensional)

Auch solche mehrdimensionalen Arrays lassen sich in Java abbilden. Dabei werden

- so viele eckige Klammerpaare verwendet
- wie die Dimension des Array ist

Beispiele:

Ein 2-dimensionales Array von 8 mal 8 Integer-Werten (beispielsweise beim Schach) lässt sich wie folgt definieren:

```
int[ ][ ] schachbrett = new int[8][8];
```

Ein 3-dimensionales Array von 1024 mal 1024 mal 1024 Messwerten (beispielsweise für die Computertomografie) lässt sich wie folgt definieren:

```
double[ ][ ][ ] aufnahme = new double[1024][1024][1024];
```

Auf die Elemente in einem mehrdimensionalen Array wird dem entsprechend auch zugegriffen, beispielsweise in einem 2-dimensionalen Array über die

```
arrayName[index1][index2]
```

Beispielcode

```
schachbrett[2][4] = 2;
```

Übungsaufgaben

Arrays (leicht)

a

Schreiben Sie ein Programm, dass

- eine Zahl q einliest
- und danach
 - q Messwerte einliest
- und danach
 - alle q Messwerte ausgibt
 - den Durchschnitt der q Messwerte ausgibt

Arrays

b

Schreiben Sie ein Programm, dass

- ein 8-mal-8 Array definiert
- immer wieder folgendes macht:
 - eine Zahl x, eine Zahl y und eine Zahl w einliest
 - das Array an der Stelle [x][y] auf w setzt
 - das Array dann ausdruckt, und zwar als 8x8-Quadrat

Einige einfache Programme

Raten einer Zahl

Es soll ein Programm geschrieben werden, dass sich

- “zufällig” eine Zahl **zufall** “ausdenkt”
- der Benutzer soll so lange eine Zahl **geraten** eingeben, bis **geraten** gleich **zufall** ist
- dabei soll das Programm “*zu klein*” bzw. “*zu gross*” ausgeben, wenn **zufall** nicht richtig geraten wurde

Erzeugung einer Zufallszahl:

Eine Zufallszahl **zufall** zwischen 0 und 999 (jeweils einschliesslich) beispielsweise lässt sich durch folgenden Code erzeugen:

```
int zufall = (int)(Math.random()*1000);
```

Kurze Aufgabe:

Wie lässt sich also eine Zufallszahl zwischen 1 und 1000 (jeweils einschliesslich) erzeugen??

Einschub - Verlassen einer Schleife: *while(nichtGefunden)* vs. *while(true)-break*

Oft wird eine Schleife verwendet,

- um einen bestimmten Wert zu finden
- und wenn dieser Wert gefunden wird,
- soll die Schleife verlassen werden

Beispielweise, wenn der kleinste Teiler einer Zahl gesucht wird.

Zur Lösung dieser Problemstellung gibt es zwei "Grundschemata":

1. Verwendung einer booleschen Variablen

```
boolean gefunden = false;
while(gefunden==false)
{
    if(... "TestNachDemWert")
    { gefunden = true;
    }
    else
    {
        ... "RestDesProgrammes"
    }
}
```

2. Verwendung einer "Endlosschleife" mit "break"

```
while(true) // an sich eine "Endlosschleife"  
{  
    if(... "TestNachDemWert")  
    { break;  
    }  
    ... "RestDesProgrammes"  
}
```

Die Suche nach dem kleinsten Teiler einer Zahl könnte nach Variante 1 wie folgt programmiert werden:

```
int zahl = 247;
```

```
int teiler = 2;
```

```
boolean gefunden = false;
```

```
while(gefunden==false)
```

```
{
```

```
    if( (zahl%teiler) ==0)
```

```
    {    gefunden = true;
```

```
    }
```

```
    else
```

```
    {
```

```
        teiler = teiler+1;
```

```
    }
```

```
}
```

```
// gefundenen Wert ausdrucken:
```

```
System.out.println("kleinster Teiler ist: "+teiler);
```

Nach Variante 2 könnte wie folgt programmiert werden:

```
int zahl = 247;
int teiler = 2;

while(true)
{
    if( (zahl%teiler) ==0)
    { break;
    }

    teiler = teiler+1;
}

// gefundenen Wert ausdrucken:
System.out.println("kleinster Teiler ist: "+teiler);
```

Normalerweise wird in der Informatik

- die Variante 1 als "sauberer" angesehen
- Variante 2 empfinde ich selber oft als "knackiger"

Übungsaufgaben

Schleifen / Abbruch

a

Schreiben Sie ein Programm für das "Raten einer Zahl"!

Dabei soll sich

- der Computer eine Zahl zwischen 1 und 1000 ausdenken
- der Benutzer immer wieder eine geratene Zahl eingeben
- der Computer sagen, ob die geratene Zahl zu klein, zu gross oder korrekt war
- wenn die Zahl korrekt war, soll der Computer schreiben "gefunden", ausgeben, wie oft der Benutzer raten musste und das Programm beenden.

Sortieren von Arrays

Manchmal passiert es, dass (z.b. für Statistiken) ein Array von Werte sortiert werden muss. Dazu soll ein einfacher (allerdings sehr langsamer) Algorithmus vorgestellt werden.

Bemerkung: Tauschen von zwei Variablen

Sollen die Inhalte zweier Variablen getauscht werden, **so ist dazu eine dritte, temporäre Variable notwendig.**

Beispiel:

Der folgende Code ist FALSCH:

```
int a = 4;
int b = 12;
// falsch !!!
a = b;
b = a;
```

Frage:

Was haben a und b nach dem Programmstück für Werte?

Statt dessen muss der Code wie folgt aussehen:

```
int a = 4;
int b = 12;
// richtig: mit temporärer Variable
int tmp = a;
a = b;
b = tmp;
```

Kurze Aufgabe:

Verdeutlichen Sie sich selber auf dem Papier, wie der Tausch hier funktioniert!

Sortieren eines Arrays – wiederholtes Tauschen

Hierbei soll ein Array von n nicht sortierten Zahlen vorliegen (also mit den Indizes von 0 bis $n-1$).

Die Grundidee zum Sortieren ist dabei folgende:

- solange im Array zwei Zahlen nebeneinander "falsch sind"
- (also eine grössere Zahl links neben einer kleineren steht)
- werden diese beiden Zahlen vertauscht
- und der Vorgang wiederholt
- dabei wird nach einem solchen Paar immer wieder von links nach rechts gesucht

Beispiel:

Das folgende Array mit 5 Zahlen würde wie folgt sortiert werden:

4 8 1 3 5

4 1 8 3 5

1 4 8 3 5

1 4 3 8 5

1 3 4 8 5

1 3 4 5 8

Der Algorithmus dazu sähe wie folgt aus, wenn eine Schleife mit **break** verwendet wird:

Mache immer wieder folgendes:

nimm an, das Array sei sortiert

mach für alle Paare $(i, i+1)$ folgendes:

WENN das Paar "falsch herum" ist:

DANN: ist das Array doch noch nicht sortiert

tausche das Paar $(i, i+1)$

WENN doch alle Paare sortiert waren

DANN: verlasse die Schleife

Gib das gesamte Array aus

Das Java-Programm dazu sieht wie folgt aus:

```
// Beispieldaten...
int anzahl = 5;
int[] zahlen = new int[anzahl];
zahlen[0] = 4;
zahlen[1] = 8;
zahlen[2] = 1;
zahlen[3] = 3;
zahlen[4] = 5;

// Sortier-Algorithmus -----

// pruefe immer wieder, ob ein "falsches Paar" vorliegt
// wenn ja, tausche es und wiederhole die Schleife
while(true)
{
    // nimm (mal wieder...) an, alle Paare stimmen:
    boolean sortiert = true;
```

```
// finde das erste Paar, das nicht sortiert ist:
for(int i=0; i<anzahl-1; i++)
{
    if(zahlen[i]>zahlen[i+1])
    {
        // nein, es stimmen DOCH NICHT alle Paare:
        sortiert = false;

        // Paar wird getauscht:
        int tmp = zahlen[i];
        zahlen[i] = zahlen[i+1];
        zahlen[i+1] = tmp;
    }
} // for

// wenn kein "falsches Paar" gefunden wurde,
// wird die Schleife verlassen:
if(sortiert==true)
{ break;
}
} // while

// alle Zahlen ausgeben:
for(int i=0; i<anzahl; i++)
{ System.out.print(" "+zahlen[i]);
}
System.out.println();
```

Übungsaufgaben

Schleifen

Schreiben Sie ein Programm, dass für die Funktion

$$f(x) = x^3 - 2x^2 + 5x$$

a

Einliest:

- einen Startwert x_1
- einen Endwert x_2
- eine Anzahl n

und dann eine Tabelle mit den Funktionswerten von $f(x)$ ausgibt.

Dabei soll x von x_1 nach x_2 laufen, und zwar in n Schritten

Minimum/Maximum

b

Erweitern Sie ihr Programm so, dass es zusätzlich am Ende der Tabelle

- DEN Wert von x ausgibt,
- an dem der Betrag $|f(x)|$ minimal war
- $f(x)$ soll dabei auch noch ausgegeben werden

Sortierten (schwer!!)

c

Schreiben Sie einen Sortieralgorithmus und implementieren Sie ihn in Java, der wie folgt vorgeht:

- im Array wird ab der Stelle 0 der maximale Wert gesucht
- dieser Wert tauscht mit dem Wert an der Stelle 0
- dann wird ab der Stelle 1 der nächst-maximale Wert gesucht
- dieser Wert tauscht mit dem Wert an der Stelle 1
- dann wird ab der Stelle 2 der usw

Objektorientierung

Seit Mitte der 90-er Jahre ist *Objektorientierung* ein sehr wichtiges Konzept in der Softwareerstellung.

Dabei ist Objektorientierung zuerst einmal

- ein Konzept, das die Informatik aus den Geisteswissenschaften übernommen hat
- und eine Art und Weise, die Welt zu sehen und in Programmen "nachzubilden"

Objektorientierung geht dabei aus

dass die Welt aus einer Menge von Objekten besteht,

und jedes Objekt hat

- bestimmte, für dieses Objekt typische Eigenschaften
- die Möglichkeit, bestimmte Aktionen auszuführen
- die Möglichkeit, bestimmte Berechnungen durchzuführen

Beispiele:

Ein Hund hat

- einen bestimmten Namen
- einen Besitzer
- kann bellen
- kann ein Stöckchen zurückholen

Ein Läufer in einem Marathonlauf (der von einem Programm verwaltet werden soll) hat

- einen bestimmten Namen
- ein bestimmtes Alter
- eine bestimmte Startnummer
- erreicht das Ziel nach einer bestimmten, für ihn typischen Zeit

Ein ZeichenObjekt (das wir definieren wollen) soll folgendes können:

- eine Linie mit n Sternen zeichnen
- ein n-mal-m grosses Rechteck zeichnen
- eine gleichschenkliges rechtwinkliges Dreieck der Seitenlänge n zeichnen

Klassen und Objekte

Verschiedene Objekte, die aber grundsätzlich sehr ähnlich sind, werden in sogenannte Klassen eingeteilt. So können wir uns vorstellen, dass folgende Klassen existieren:

- die Klasse aller Hunde
- die Klasse aller Marathonläufer
- die Klasse aller ZeichenObjekte

Die Definition einer solchen Klasse in Java geschieht mittels des Schlüsselwortes "class":

class

Das Schlüsselwort `class` dient in Java dazu,

- eine Klasse zu definieren,
- die Objekte mit ähnlichen Eigenschaften und ähnlichem Verhalten enthält

Benutzung:

```
public class KlassenName
{
    ... Definition der Instanzvariablen ...
    ... Definition der Methoden ...
}
```

Dabei sind die

- Instanzvariablen die "Eigenschaften"
 - Methoden die möglichen "Aktionen"
- eines Elementes der Klasse

Der Name einer Klasse ist dabei immer

- ein Substantiv
- im Singular (!!)

Beispiel:

Ein erster Schritt zur Definition der Klasse **ZeichenObjekt** würde wie folgt aussehen:

```
public class ZeichenObjekt
{
    ... mehrHierzuSpäter ..
}
```

Eine Klasse **Hund** würde wie folgt anfangen:

```
public class Hund
{
    ...mehrAuchHierSpäter
}
```

Methoden

Wie schon erwähnt, kann ein Objekt einer bestimmten Klasse

- bestimmte Aktionen ausführen und/oder
- bestimmte Berechnungen durchführen

Dies geschieht,

- indem in der zugehörigen Klasse
- bestimmte **Methoden** definiert werden

Eine Methode ist also

- eine Aktion,
- die jedes Objekt dieser Klasse durchführen kann

Bisher haben wir Aktionen in Java-Programmen durchgeführt, indem wir den entsprechenden Code in die main-Methode geschrieben haben.

So wird in der Objektorientierten Programmierung aber an sich **nicht** vorgegangen!!

Statt dessen werden

- zu einer Klasse bestimmte Methoden definiert
- und dort wird der auszuführende Code hingeschrieben

Die Definition von Methoden in einer Java-Klasse geschieht dabei wie folgt:

Methode

Eine Methode ist

- eine Aktion in einer bestimmten Klasse,
- die jedes Objekt dieser Klasse durchführen kann.

Benutzung:

Innerhalb einer Klasse wird eine Methode wie folgt definiert:

```
public RückgabeTyp MethodenName( ParameterListe )
{
    ... CodeDerMethode ...
}
```

Der Rückgabebetyp ist dabei

- ein Typname wie `int`, `double`, `String` oder ähnliches, wenn die Methode etwas im mathematischen Sinne "berechnet"
- das Schlüsselwort `"void"`, wenn die Methode nichts berechnet, sondern nur Aktionen ausführt

Die ParameterListe

- besteht dabei aus Paaren Typname/Parametername, wenn Parameter vorhanden sind. Die Paare werden durch Kommas getrennt
- ist "leer", also einfach `"( )"`, wenn keine Parameter vorhanden sind

Namen von Methode

- werden immer kleingeschrieben
- fangen mit einem Verb an

Beispiel

Schauen wir uns beispielsweise die von uns zu definierende Klasse **ZeichenObjekt** an.

Ein solches ZeichenObjekt soll zuerst einmal aus "Sternchen"

- eine 5 Sterne breite Linie zeichnen können
- ein Quadrat der Kantenlänge 7 zeichnen können

Die Definition einer Klasse **ZeichenObjekt** sähe dann wie folgt aus:

```
public class ZeichenObjekt
{
    public void zeichneLinie( )
    {
        System.out.println("* * * * *");
    }

    public void zeichneQuadrat()
    {
        for(int i=0; i<7; i++)
        {
            for(int k=0; k<7; k++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Erzeugen von Objekten einer Klasse

Wir haben jetzt definiert, wie die Klasse "ZeichenObjekt" aussehen soll. Was wir aber noch nicht haben, ist ein Objekt dieser Klasse.

Ähnlich ist es bei Objekten der realen Welt: wir alle

- wissen grundsätzlich, wie "ein Hund" aussieht und funktioniert (= Klasse),
- wenn wir aber einen bestimmten Hund haben wollen (= Objekt),
- müssen wir uns erst einen beschaffen (zB. Im Tierheim oder aus dem Tiergeschäft)

Ähnlich ist es bei unserer Klasse **ZeichenObjekt**:

- die Klasse haben wir definiert,
- wir möchten jetzt gerne auch ein ZeichenObjekt "kaufen", um mit ihm ein wenig zu spielen

Das Erzeugen von Objekten zu einer bestimmten Klasse geschieht in Java durch den Befehl `new`:

new (Erzeugen von Objekten)

Der Befehl **new** dient in Java dazu,

- zu einer bestimmten Klasse
- ein Objekt dieser Klasse zu erzeugen

Benutzung:

Mit dem Befehl new wird

- ein neues Objekt erzeugt
- und (typischerweise) einer Variablen zugeordnet,

indem folgendes Schema verwendet wird:

KlassenName Variable = new KlassenName();

Beispiel:

Wir können ein neues ZeichenObjekt erzeugen und einer Variablen zuordnen, indem wir folgenden Code verwenden:

ZeichenObjekt zo = new ZeichenObjekt();

Grafisch:

Ansprechen von Methoden eines Objektes

Nach der Erzeugung eines Objektes zu einer Klasse

- besitzen wir also eine Variable,
- über die wir die Methoden des Objektes aufrufen können

Dies geschieht, indem wir die Methoden des Objektes über den sogenannten Punkt-Operator aufrufen:

■ (Punkt-Operator)

Der Punkt-Operator • dient in Java dazu,

- zu einem bestimmten Objekt
- das einer Variablen zugeordnet ist
- eine Methode des Objektes aufzurufen

Benutzung:

Eine Methode eines Objektes wird aufgerufen, indem folgendes Schema verwendet wird:

variablenName . methodenName(Parameter);

Beispiel

Wir haben vorhin durch folgenden Code ein ZeichenObjekt erzeugt:

```
ZeichenObjekt zo = new ZeichenObjekt();
```

Jetzt können wir die Methode "zeichneQuadrat" dieses Objektes durch folgenden Code aufrufen:

```
zo.zeichneQuadrat();
```

Oder wir können das Objekt eine Linie zeichnen lassen durch

```
zo.zeichneLinie();
```

Der obige Code muss natürlich wiederum in der main-Methode der Klasse stehen, so dass die gesamte Klasse dann wie folgt aussieht:

```
// Code der Klasse ZeichenObjekt:
//

public class ZeichenObjekt
{
    public void zeichneLinie(int n)
    {
        for(int x=0; x<n; x++)
        { System.out.print("+ ");
          }
        System.out.println();
    }

    public void zeichneQuadrat(int s)
    {
        for(int i=0; i<s; i++)
        {
            for(int k=0; k<s; k++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args)
    {
        ZeichenObjekt x = new ZeichenObjekt();

        x.zeichneQuadrat(6);
        x.zeichneLinie(11);
    }
}
```

übungsaufgaben

a

Implementieren und testen Sie die Klasse **ZeichenObjekt**

b

Schreiben Sie eine Klasse **Hund** , die eine Methode **belle()** anbietet.

Bei Aufruf dieser Methode soll auf dem Bildschirm der Text "wuff" erscheinen.

c

Schreiben sie analog dazu eine Klasse **Katze**, die eine Methode "**miaue()**" anbietet

d

Erweitern Sie die Klasse **ZeichenObjekt** durch eine Methode **zeichneDreieck()**, die ein Dreieck der Kantenlänge 5 auf den Bildschirm zeichnet.

e

Schreiben Sie eine Klasse **Quadrierer**.

Diese Klasse soll eine Methode **berechneQuadrate()** anbieten, die

- eine Zahl n einliest (über einen **Leser**)
- alle Quadrate von 1 bis n auf dem Bildschirm ausgibt

Parameter von Methoden

Bisher wurden alle Methoden in Objekten einfach aufgerufen, ohne dass ihnen dabei weitere Informationen mitgegeben wurden.

Methoden von Objekten können aber beim Aufruf auch Informationen mitgegeben werden.

Diese Informationen **werden dann *Parameter* genannt.**

Damit eine Methode Parameter annehmen kann, muss Sie in der entsprechenden Klasse etwas "erweitert" definiert werden, **indem sie durch eine**

***Parameterliste* erweitert wird:**

Parameter in Methoden

Wenn einer Methode Parameter mitgegeben werden sollen, wird folgende Schreibweise verwendet:

```
public Rückgabetyp Methodenname(Parameterliste)
{
    ...
}
```

Dabei ist der Rückgabetyp wiederum

- entweder "void" (erst einmal)
- oder ein gültiger Datentyp

Die Parameterliste hat folgendes Aussehen:

- Typ Parametername
- Typ Parametername , Typ2 Parametername2
- Typ Parametername,
Typ2 Parametername2,
Typ3 Parametername3
- usw...

Die Parameter können dann in der Methode wie normale Variablen verwendet werden.

Beispiele

Eine Klasse **Berechner** mit einer Methode **druckeQuadratVon (. . .)** würde wie folgt definiert werden:

```
public class Berechner
{
    public void druckeQuadratVon(double x)
    {
        double quadrat = x*x;
        System.out.println("Quadrat von " + x + " ist:" + quadrat);
    }
    . . .
}
```

Eine zusätzliche Methode **druckeAHochB (. . .)** würde in dieser Klasse dann wie folgt definiert werden:

```
public void druckeAHochB(double a, int b)
{
    double result = 1.0;
    for(int i=0; i<b; i++)
    {
        result = result*a;
    }

    System.out.println(" "+a+" hoch "+b+" ist: "+result);
}
```

Eine main-Methode zum Testen der obigen Methoden sähe z.B. wie folgt aus:

```
public static void main(String[] args)
{
    Berechner b = new Berechner();

    b.druckeQuadratVon(4.1);
    b.druckeAHochB(2.1, 3);
}
```

Aber nicht nur primitive Datentypen wie Zahlen können einer Methode übergeben werden, sondern einer Methode kann jede beliebige Art von Datentyp übergeben werden.

Beispiel

Die folgende Methode druckt einen beliebigen Text **zeile** genau **n**-mal:

```
public void druckeNmal(String zeile, int n)
{
    for(int i=0; i<n; i++)
    {
        System.out.println(zeile);
    }
}
```

Aufgerufen würde diese Methode beispielsweise so:

```
b.druckeNmal("eine kleine Zeile", 7);
```

Oder wir könnten in der Klasse **ZeichenObjekt** die Methode **zeichneQuadrat(...)** verbessern, so dass sie die Information bekommt, wie gross die Seitenlänge des zu zeichnenden Quadrates ist:

```
public void zeichneQuadrat(int breite)
{
    for(int y=0; y<breite; y++)
    {
        for(int x=0; x<breite; x++)
        {
            System.out.print(" * ");
        }
        System.out.println();
    }
}
```

Übungsaufgaben

a

Schreiben Sie eine Klasse **TabellenZeichner** und darin eine Methode

```
zeigeQuadratVonBis(int von, int bis)
```

die die Quadrate von **von** bis **bis** als Tabelle darstellt.

Beispiel: der Aufruf von

```
TabellenZeichner tz =  
    new TabellenZeichner();  
tz.zeigeQuadratVonBis(5,9);
```

würde folgendes schreiben:

```
5  25  
6  36  
7  49  
8  64  
9  81
```

b

Erweitern Sie die Klasse **ZeichenObjekt** um eine Methode

```
zeichneRechteck(int hoehe, int breite)
```

die ein Rechteck der entsprechenden Hoehe und Breite darstellt

(sehr schwierig)

Erweitern Sie die Klasse **ZeichenObjekt** um eine Methode

```
zeichneDreieck(int hoehe, int breite)
```

die ein Dreieck der entsprechenden Hoehe und Breite darstellt

c

(etwas
schwierig)

Erweitern Sie die obige Klasse **TabellenZeichner** um eine Methode

```
zeigeQuadratischeGleichung(  
    double a, double b, double c,  
    double start, double ende, double schritt)
```

d

die in einer Tabelle

- die Werte von ax^2+bx+c

- in den Grenzen von **start** bis **ende**
- mit der Schrittweite **schrift** darstellt

Methoden: Prozeduren vs. Funktionen

Unsere bisherigen Methoden

- hatten Parameter oder auch nicht
- und haben dann Aktionen durchgeführt (oft etwas ausgedruckt)
- haben aber - im mathematischen Sinne - keinen Wert zurückgeliefert wie zum Beispiel eine mathematische Funktion wie der Sinus oder der Logarithmus

Solche Methoden werden Prozeduren genannt:

Prozedur

Eine Prozedur ist eine Methode, die

- gegebenenfalls Parameter besitzt (oder auch keine)
- Aktionen durchführt
- aber keinen Rückgabewert zurückliefert
- also auch kein *return*-Befehl benötigt

Prozeduren werden wie folgt definiert:

```
public void Methodenname(Parameterliste)
{
    ...
}
```

Im Gegensatz dazu stehen Funktionen:

Funktion

Eine Funktion ist eine Methode, die

- meist ein oder mehrere Parameter besitzt
- (mathematische) Berechnungen durchführt
- und einen Rückgabewert zurückliefert
- der mit dem *return*-Befehl zurückgegeben wird

Funktionen werden wie folgt definiert:

```
public Rückgabotyp Methodenname(Parameterliste)  
{  
    ...  
    return Rückgabewert;  
}
```

Funktionen werden meistens

- in Programmen verwendet,
- um mathematische oder technische (Zwischen-)Berechnungen durchzuführen
- und den berechneten Wert zurückzuliefern

Beispiel

Im folgenden soll eine Klasse **MatheObjekt** definiert werden, die verschiedene mathematische Berechnungen durchführen kann:

```
public class MatheObjekt
{
    public double quadrat(double x)
    {
        double result = x*x;
        return result;
    }

    public double hochDrei(double n)
    {
        double ergebnis = n*n*n;
        return ergebnis;
    }
}
```

Eine main-Methode zum Testen dieser Klasse sähe z.B. wie folgt aus:

```
MatheObjekt m = new MatheObjekt();
```

```
double a = m.quadrat(5.1);
```

```
System.out.println(a);
```

```
double b = m.hochDrei(5.1);
```

```
System.out.println(b);
```

Funktionen können aber auch komplexere Arbeiten bewältigen. Die folgende Methode z.B. gibt das Maximum aller Zahlen in einem Array zurück (zusätzlich in der Klasse MatheObjekt...):

```
public double maximum(double[] werte)
{
    // herausfinden, wie lang das Array ist:
    int laenge = werte.length;

    // Maximum finden
    double maximum = werte[0];
    for(int i=1; i<laenge; i++)
    {
        if(werte[i]>maximum)
        { maximum = werte[i];
        }
    }

    // Maximum zurückgeben
    return maximum;
}
```

Code in der main-Methode zum Testen sähe z.B. wie folgt aus:

```
m = new MatheObjekt();

double test[] = new double[5];
test[0] = 4;
test[1] = 1;
test[2] = 8;
test[3] = 5;
test[4] = 2;

double max = m.maximum(test);
System.out.println("Maximum ist: "+max);
```

übungsaufgaben

Schreiben Sie eine Klasse **Potenzierer**, die folgende **Funktionen** anbietet:

- `public double aHochB(double a, int b)`
- `public double zweiHochX(double x)`

a

Schreiben Sie in der Klasse **MatheObjekt** eine Funktion

`public double mittelwert(double[] werte)`

die ein Array von Werten bekommt und den Mittelwert dieser Werte zurück gibt

b

Objekte und Eigenschaften

Bisher waren Objekte für uns einfach eine Art von “Behälter”, um Methoden zu enthalten und damit zu “ordnen”.

Der Begriff des Objektes ist in der Objektorientierten Programmierung jedoch viel weiter gespannt:

Objekte in der Objektorientierten Programmierung dienen dazu,

- bestimmte Eigenschaften besitzen zu können,
- und bestimmte Aktionen
- (gegebenfalls eben auf diesen Eigenschaften)
- ausführen zu können.

Damit können Objekte unter anderem dazu dienen,

- “reale” Objekte aus der wirklichen Welt
- modellieren und programmieren zu können.

Beispiel

Ein Mensch aus der “realen” Welt hat unter anderem die folgenden Eigenschaften:

- einen Vornamen
- einen Namen
- ein Alter
- ggf. einen Beruf

Für unser Beispiel soll er es z.B. erlauben

- Geburtstag zu haben (wobei sein Alter um 1 erhöht wird)
- seinen Nachnamen zu ändern (z.B. bei einer Heirat)
- ausgedruckt zu werden

Instanzvariablen

Eigenschaften von Objekten einer Klasse werden durch sogenannte "Instanzvariablen" realisiert.

Instanzvariable

Eine Instanzvariable ist

- eine Variable, die in der Klasse des Objektes definiert wird
- und für jedes Objekt einen eigenen Wert haben kann
- auf Instanzvariablen kann nur das Objekt selber zugreifen
- von ausserhalb des Objektes kann nicht auf sie zugegriffen werden

Instanzvariablen werden wie folgt definiert:

```
public class KlassenName
{
    private Datentyp1 Variable1;
    private Datentyp2 Variable2;
    private Datentyp3 Variable3;
    ...
    ... wie vorher (Methoden)...
}
```

Auch Instanzvariablen kann ein Startwert zugewiesen werden. Dann lautet die Schreibweise natürlich

`private Datentyp Variable = Startwert;`

Beispiel

Eine Klasse "Mensch" würde im ersten Schritt wie folgt modelliert werden:

```
public class Mensch
{
    private String vorname;
    private String nachname;
    private int    alter;
    private String beruf = "nicht bekannt";

    public void drucke()
    {
        System.out.println("Name: "+vorname+" "+nachname);
        System.out.println("Alter: "+alter);
        System.out.println("Beruf: "+beruf);
    }
}
```

Eine main-Methode zum Testen sähe z.b. wie folgt aus:

```
Mensch m = new Mensch();
m.drucke();
```

Interessanterweise ist die Ausgabe dieses Programmes:

Name: null null

Alter: 0

Beruf: nicht bekannt

null

Der Wert **null** dient in Java dazu

- den Wert einer Variablen als "ungültig" bzw. "nicht bekannt" zu beschreiben
- null kann an Variablen aus jeder Klasse zugewiesen werden
- nicht jedoch primitiven Datentypen (int, long, double, boolean, char, ...)

private (Datenkapselung)

Wie oben beschrieben, kann der Wert einer Instanzvariablen nicht von ausserhalb eines Objektes geändert werden,

- nur das Objekt selber kann seine Instanzvariablen ändern

(an sich: seine Klasse kann dies, aber mehr dazu später...)

Dieses Prinzip wird als Datenkapselung bezeichnet:

Datenkapselung (data hiding)

Als Datenkapselung (data hiding) wird in der Objektorientierten Programmierung das Prinzip bezeichnet

- dass ein Objekt Instanzvariablen besitzen kann
- aber nur das Objekt selber auf diese Variablen zugreifen kann
- von aussen sind die Instanzvariablen "geschützt", auf sie kann nicht zugegriffen werden
- Dies wird in Java durch das Schlüsselwort "private" definiert.

Wenn von aussen auf die Variablen zugegriffen werden soll,

- so muss die Klasse des Objektes
- dafür entsprechende Methoden bereitstellen

Beispiel

Sollen Name, Vorname, Alter und Beruf von Objekten der Klasse "Mensch" geändert werden sollen, so muss die Klasse dafür z.B. folgende Methoden bereitstellen:

```
public void setzeVorname(String v)
{   vorname = v;
}
```

```
public void setzeNachname(String n)
{   nachname = n;
}
```

```
public void setzeAlter(int a)
{   alter = a;
}
```

Jetzt kann folgender Code ausgeführt werden (...main...):

```
Mensch m = new Mensch();
m.setzeVorname("Homer");
m.setzeNachname("Simpson");
m.setzeAlter(36);
m.drucke();
```

Das Ergebnis ist dem entsprechend

Name: Homer Simpson

Alter: 36

Beruf: nicht bekannt

Jetzt lässt sich natürlich auch beispielsweise eine Methode zur Beschreibung des Vorgangs "hat Geburtstag" schreiben:

```
public void hatGeburtstag()  
{ alter = alter+1;  
}
```

folgender Code

```
Mensch m = new Mensch();  
m.setzeVorname("Homer");  
m.setzeNachname("Simpson");  
m.setzeAlter(36);  
m.hatGeburtstag();  
m.drucke();
```

hat dann auch folgende Ausgabe:

```
Name: Homer Simpson  
Alter: 37  
Beruf: nicht bekannt
```

Konstruktor - Erzeugung von Objekten

Die obige Vorgehensweise,

- einen “leeren” Menschen zu erzeugen
 - und ihm “danach” einen Vornamen, einen Namen und ein Alter zuzuweisen
- ist sehr **unelegant**.

An sich nämlich

- besitzt jeder Mensch
- einen Vorname, einen Nachnamen und ein Alter,
- (jedoch hat nicht jeder Mensch einen Beruf)

Daher wäre es deutlich eleganter, einem Objekt der Klasse “Mensch”

- schon bei seiner Erzeugung
- einen Vornamen, einen Nachnamen und ein Alter
- zuzuweisen

Dies kann durch einen sogenannten Konstruktor geschehen:

Konstruktor

Ein Konstruktor ist

- eine Methode
- die automatisch bei der Erzeugung eines Objektes mit new aufgerufen wird,
- der die "Grunddaten" des Objektes mitgegeben werden,
- und die die Aufgabe hat, den Instanzvariablen des neuen Objektes
- diese Daten zuzuweisen

Die Schreibweise des Konstruktors einer Klasse lautet

```
public Klassenname ( Datentyp1 Variable1 ,  
                     Datentyp2 Variable2, ... )
```

Existiert ein Konstruktor zu einer Klasse,

so können Objekte der Klasse nur noch über diesen Konstruktor erzeugt werden

Wichtig!!!

der Konstruktor einer Klasse hat gar keinen Rückgabetyt, also noch nicht einmal void

Beispiel

Da jeder Mensch Vorname, Nachname und Alter besitzt, sollte der Konstruktor der Klasse "Mensch" wie folgt aussehen:

```
public Mensch(String v, String n, int a)
{
    vorname   = v;
    nachname  = n;
    alter     = a;
}
```

Damit können natürlich

- die Methoden `setzeVorname`, `setzeNachname` und `setzeAlter` von vorhin
- vollständig entfallen, da sie nicht mehr benötigt werden

Die main-Methode sieht dann einfach wie folgt aus:

```
Mensch m = new Mensch("Homer", "Simpson", 36);
m.hatGeburtstag();
m.drucke();
```

Das Ergebnis ist weiterhin

```
Name: Homer Simpson
Alter: 37
Beruf: nicht bekannt
```

Die gesamte Klasse sieht jetzt wie folgt aus:

```
public class Mensch
{
    private String vorname;
    private String nachname;
    private int    alter;
    private String beruf = "nicht bekannt";

    public Mensch(String v, String n, int a)
    {
        vorname  = v;
        nachname = n;
        alter    = a;
    }

    public void drucke()
    {
        System.out.println("Name: "+vorname+" "+nachname);
        System.out.println("Alter: "+alter);
        System.out.println("Beruf: "+beruf);
    }

    public void hatGeburtstag()
    {
        alter = alter+1;
    }

    // Zum Testen:
    //
    public static void main(String[] args)
    {
        Mensch m = new Mensch("Homer", "Simpson", 36);
        m.hatGeburtstag();
        m.drucke();
    }
}
```

übungsaufgaben

Schreiben Sie selber die Methode **setzeBeruf** in der Klasse **Mensch**

a

Schreiben Sie selber die Methode **aendereNachname** in der Klasse **Mensch**

b

Erweitern Sie selber die Klasse **Mensch** um eine Instanzvariable **Gewicht**

c

Schreiben Sie eine Methode **setzteGewicht** und ändern Sie die Methode **drucke** entsprechend ab

d

Erweitern Sie selber die Klasse **Mensch** um eine Instanzvariable **anzahlKinder** und ergänzen Sie die Klasse um die ihrer Meinung nach dabei notwendige(n) Methode(n)

Beispiel: 3-dimensionale Vektoren

Als weiteres, eher “technisches” Beispiel soll hier dargestellt werden, wie 3-dimensionale Vektoren aus der Mathematik in Java implementiert werden können.

Ein Vektor mit den 3 Komponenten 1.0, 2.5 und 3.2 zum Beispiel wird bekannterweise in der Mathematik in einer der folgenden Schreibweisen dargestellt:

Wir gehen nun in folgendem “7-Schritte-Schema” vor:

Schritt 1: Benennung der Klasse

Ein guter Name für die Klasse “3-dimensionaler-Vektor” ist z.B. “Vektor3d”

Schritt 2: Definition der Instanzvariablen

Jeder 3-dimensionaler Vektor

- hat 3 Komponenten x, y und z
- die jeweils Fließkommazahlen sind

Damit sieht unsere Klasse bisher wie folgt aus:

```
public class Vector3d
{
    // Instanzvariablen: jeder Vektor hat
    // eine x, y, und z-Komponente
    private double x;
    private double y;
    private double z;
}
```

Schritt 3: Konstruktor

Zu jeder Klasse **muss überlegt werden, wie ein passender Konstruktor aussehen sollte**

Zu jedem Vektor3d gehört unbedingt die Information,
welche Werte seine Instanzvariablen x, y und z besitzen

Damit sieht der Konstruktor der Klasse wie folgt aus:

```
// Konstruktor
public Vektor3d(double a, double b, double c)
{
    x=a;
    y=b;
    z=c;
}
```

Schritt 4: print-Methode

Zu jeder Klasse muss weiterhin überlegt werden, **wie ein Objekt dieser Klasse ausgedruckt werden kann**

Dies definieren wir, **indem wir eine print-Methode für die Klasse implementieren.**

Unsere Vektoren (x,y,z) drucken wir in der folgenden Form aus:

Damit sieht die print-Methode wie folgt aus:

```
public void print()  
{  
    System.out.println("( "+x+" "+y+" "+z+" )");  
}
```

Schritt 5: erster Test

nach Schritt 1 bis 4 können wir schon einmal testweise

ein Objekt der Klasse erzeugen
und das Objekt ausdrucken

Um dies auszuprobieren, implementieren wir folgende main-Methode:

```
public static void main(String[] args)
{
    // Vektor v erzeugen, Vektor ausdrucken
    System.out.println("Vektor v: ");
    Vektor3d v = new Vektor3d(3, 4, 2);
    v.print();
}
```

Scheint zu funktionieren...

Schritt 6: Definition der Methoden der Klasse

Wir können jetzt

- Objekt der Klasse erzeugen
- diese Objekte ausdrucken

Jetzt ist der Zeitpunkt gekommen, **die notwendigen Methoden der Klasse zu implementieren**

Methode 1: Länge des Vektors

Unsere erste Methode soll die Funktion "Länge des Vektors" sein. Die Länge eines Vektors (x,y,z) ist bekanntermassen:

Wir stellen dabei fest:

- ein guter Name für diese Methode ist "laenge"
- diese Methode ist eine Funktion, da sie einen Rückgabewert berechnet
- dieser Rückgabewert ist eine Fließkommazahl

Damit fängt diese Methode wie folgt an:

```
public double laenge()  
{  
    ...  
}
```

Da die Länge des Vektors die Wurzel der Summe der Quadrate ist, sieht die gesamte Methode dann wie folgt aus:

```
public double laenge()  
{  
    double summe = x*x + y*y + z*z;  
    double wurzel = Math.sqrt(summe);  
    return wurzel;  
}
```

Test der Methode

Wir erweitern jetzt die main-Methode um folgende Zeile, um zu testen, ob die Längenfunktion korrekt implementiert ist:

```
System.out.println("Laenge von v: " + v.laenge());
```

Scheint zu funktionieren...

Methode 2: Skalarprodukt

Unsere zweite Methode soll die Funktion "Skalarprodukt aus 2 Vektoren" sein. Das Skalarprodukt von 2 Vektoren v und $v2$ sieht bekanntermassen wie folgt aus:

Wir stellen dabei fest:

- ein guter Name für diese Methode ist "mal"
- die Funktion hat einen Parameter, nämlich den zu multiplizierenden Vektor $v2$
- diese Methode ist eine Funktion, da sie einen Rückgabewert berechnet
- dieser Rückgabewert ist eine Fließkommazahl

Damit fängt diese Methode wie folgt an:

```
public double mal(Vektor3d v2)
```

Einschub: Zugriff auf Instanzvariablen

Hier ist es notwendig, eine kurze Erläuterung zum Thema “data hiding” zu bringen. Wir hatten im vorigen Kapitel besprochen, dass

- Instanzvariablen grundsätzlich “private” sind,
- damit nur das Objekt selber Zugriff auf diese Variablen hat

Davon gibt es jedoch eine wichtige Ausnahme:

Zugriff auf Instanzvariablen eines Objektes der selben Klasse

- in einer Methode eines Objektes
- kann das Objekt auch auf Instanzvariablen eines anderen Objektes zugreifen
- aber nur, wenn das andere Objekt die gleiche Klasse besitzt

Die Schreibweise lautet dann

NameDesObjektes . NameDerInstanzvariable

Damit können wir die Methode "mal" zum Berechnen des Skalarproduktes wie folgt definieren:

```
public double mal(Vektor3d v2)
{
    double result = x*v2.x + y*v2.y + z*v2.z;
    return result;
}
```

Wir sehen hier: Wenn unserer Vektor den Befehl bekommt,

- "das Produkt von sich und dem anderen Vektor v2 zu berechnen"
- dann greift er auf seine Variablen x, y und z zu
- und er greift auf die Variablen x, y und z des Vektors v2 zu
- mittels der Schreibweise v2.x , v2.y und v2.z

Test der Methode

Wir erweitern jetzt die main-Methode um einen Vektor w und die Berechnung des Skalarproduktes:

```
Vektor3d w = new Vektor3d(6, 1, 5);
```

```
// v und w multiplizieren:
```

```
double s = v.mal(w);
```

```
System.out.println("v mal w: "+s);
```

Scheint zu funktionieren...

Methode 3: Kreuzprodukt

Unsere dritte Methode soll die Funktion "Kreuzprodukt aus 2 Vektoren" sein. Das Kreuzprodukt von 2 Vektoren v und v_2 sieht bekanntermassen wie folgt aus:

Wir stellen dabei fest:

- ein guter Name für diese Methode ist "kreuz"
- die Funktion hat einen Parameter, nämlich den zu multiplizierenden Vektor v_2
- diese Methode ist eine Funktion, da sie einen Rückgabewert berechnet
- dieser Rückgabewert ist selber wieder ein Vektor3d

Damit fängt diese Methode wie folgt an:

```
public Vektor3d kreuz(Vektor3d v2)
```

Nach der obigen Überlegung wird die Methode "Kreuzprodukt" dann wie folgt implementiert:

```
public Vektor3d kreuz(Vektor3d v2)
{
    double xResult = y*v2.z - z*v2.y;
    double yResult = z*v2.x - x*v2.z;
    double zResult = x*v2.y - y*v2.x;

    return new Vektor3d(xResult, yResult, zResult);
}
```

Test der Methode

Wir erweitern jetzt die main-Methode um folgende Zeile, um zu testen, ob diese Methode korrekt implementiert ist:

```
Vektor3d k = v.kreuz(w);
System.out.println("v kreuz w:");
k.print();
```

Scheint zu funktionieren...

Schritt 7: Übersicht über die gesamte Klasse

Fassen wir die bisherigen Punkte zusammen und implementieren wir noch die "Normalisierung eines Vektors", sieht unser Code wie folgt aus

```
public class Vektor3d
{
    // Instanzvariablen: jeder Vektor hat
    // eine x, y, und z-Komponente

    private double x;
    private double y;
    private double z;

    // Konstruktor

    public Vektor3d(double a, double b, double c)
    {
        x=a;
        y=b;
        z=c;
    }

    public void print()
    {
        System.out.println("( "+x+"  "+y+"  "+z+" )");
    }

    // Die Laenge eines Vektor ist die Wurzel
    // der Summe der Quadrate seiner Komponenten:
    // Die Wurzel-Methode in Java heisst Math.sqrt(...)

    public double laenge()
    {
        double summe  = x*x + y*y + z*z;
        double wurzel = Math.sqrt(summe);
        return wurzel;
    }
}
```

// "mal" ist das normale Vektorprodukt:

```
public double mal(Vektor3d v2)
{
    double result = x*v2.x + y*v2.y + z*v2.z;
    return result;
}
```

// "kreuz" ist das Kreuzprodukt:

```
public Vektor3d kreuz(Vektor3d v2)
{
    double xResult = y*v2.z - z*v2.y;
    double yResult = z*v2.x - x*v2.z;
    double zResult = x*v2.y - y*v2.x;

    return new Vektor3d(xResult, yResult, zResult);
}
```

// "normalisiert" ist ein Vektor
// "auf die Laenge 1" gebracht:

```
public Vektor3d normalisiert()
{
    double length = this.laenge();

    double xNorm = x/length;
    double yNorm = y/length;
    double zNorm = z/length;

    return new Vektor3d(xNorm, yNorm, zNorm);
}
```

```

// TEST, ob das alles auch stimmt
//

public static void main(String[] args)
{
    // Vektor v erzeugen, Vektor und Laenge ausdrucken

    System.out.println("Vektor v: ");
    Vektor3d v = new Vektor3d(3, 4, 2);
    v.print();
    System.out.println("Laenge von v: " + v.laenge());

    System.out.println();

    // Vektor w erzeugen und ausdrucken

    System.out.println("Vektor w: ");
    Vektor3d w = new Vektor3d(6, 1, 5);
    w.print();
    System.out.println("Laenge von w: " + w.laenge());

    System.out.println();

    // v und w multiplizieren:
    //

    double s = v.mal(w);
    System.out.println("v mal w: "+s);

    System.out.println();

    // Kreuzprodukt:

    Vektor3d k = v.kreuz(w);
    System.out.println("v kreuz w:");
    k.print();

    System.out.println();
}

```

```
// Kreuzprodukt normalisiert:

Vektor3d norm = k.normalisiert();
System.out.println("Kreuzprodukt normalisiert:");
norm.print();

System.out.println();

// Test, ob das Kreuzprodukt t wirklich
// Senkrecht zu v und w ist:

double p = v.mal(norm);
double q = w.mal(norm);

System.out.println("Test ob senkrecht: "+p+" "+q);
}

}
```

übungsaufgaben

a

Implementieren Sie die Klasse **Vektor3d**

b

Werden Sie sich im klaren darüber, wie diese Klasse funktioniert!!!

c

Schreiben Sie ein Programm, das

- jeweils 3 double-Zahlen einliest
- daraus einen Vektor v erstellt
- nochmal 3 double-Zahlen einliest
- daraus einen Vektor w erstellt
- dann das Vektorprodukt und das Kreuzprodukt von v und w berechnet und ausdruckt

d

Schreiben Sie eine Klasse **Matrix3d** die eine 3x3 Matrix darstellt, inclusive

- Konstruktor
- print-Methode

(schwer!!)

e

Erweitern Sie die Klasse **Matrix3d** um die Methoden

- Multiplikation mit einer zweiten Matrix
- Multiplikation mit einem Vektor3d

(sehr sehr schwer!!!)

Die String Klasse

In Java sind viele praktische Klassen schon von vorne herein eingebaut, die in der "täglichen" Programmierung effizient verwendet werden können.

Es soll hier zuerst die Klasse `String` vorgestellt werden.

Strings werden verwendet, **um Zeichenketten darzustellen, also zum Beispiel Wörter oder Sätze der natürlichen Sprache.**

Die Deklaration einer String-Variablen ist schon bekannt; sie kann mit oder ohne Zuweisung eines Initialwertes erfolgen:

```
String s;  
String w = "Winter";
```

Konkatenation

Zeichenketten können aneinander gehängt werden ("concateniert"). Dies **erfolgt einfach mittels des plus-Operators:**

```
String a = "vom Eise";  
String b = " befreit";  
String c = " sind Strom und Bäche";  
String d;  
d = a + b + c; // Konkatenation  
System.out.println(d);
```

substring

Eine der wichtigsten Operationen auf Strings **ist die Bildung von "Unterzeichen-Ketten" einer Zeichenkette.**

Diese Methode auf Strings heisst ***substring***:

substring

Sei str ein String. Dann dient die Methode

`str.substring(anfang, ende)`

dazu,

- eine "Unterzeichenkette" von str zu liefern,
- die mit dem Zeichen nummer anfang anfängt
- und ein Zeichen vor dem Zeichen nummer ende aufhört.
- Dabei fängt die Zählung der Zeichen mit 0 an !!!

Beispiel

```
String x = "Sein oder Nichtsein";  
String y = x.substring(4,10);  
System.out.println(y);
```

Ergebnis dieses Codes ist:

length

Sehr häufig muss die Länge einer Zeichenkette (also die Anzahl der Zeichen) bestimmt werden.

Dazu dient die Funktion `length`:

length

Sei `str` ein String. Dann dient die Methode

`str.length()`

dazu,

- die Länge der Zeichenkette `str` zurück zu geben,
- also die Anzahl der in `str` enthaltenen Zeichen

Beispiel

```
String x = "vom Eise befreit";  
int len = x.length();  
System.out.println(len);
```

Ergebnis dieses Codes ist:

equals

Für Vergleiche zwischen zwei Werten haben wir bisher den Vergleichsoperator "==" benutzt:

```
int x = 5;
int y = 6;
if(x == y)
{ System.out.println("beide sind gleich");
}
```

Dies geht so einfach jedoch nur **für die primitiven Datentypen.**

Wollen wir zwei Strings miteinander vergleichen, ob sie denselben Inhalt haben, so benötigen wir dafür die Methode "equals":

equals

Seien strA und strB zwei Strings. Dann dient die Methode

strA.equals(strB)

dazu,

- zu vergleichen, ob strA und strB den gleichen Inhalt haben,
- also die gleichen Zeichen
- Rückgabewert von equals ist "true" (bei Gleichheit),
- ansonsten "false"

Beispiel

```
String x = "Eis";  
String y = "Feuer";  
System.out.println(x.equals(y));
```

Ergebnis dieses Codes ist:

Einige Beispiele

Mit diesen vier Methoden lassen sich schon viele interessante Methoden erstellen; beispielsweise eine Methode, die die Anzahl der Vokale (also a,e,i,o,u) in einer Zeichenkette zählt:

```
public class TextObjekt
{
    public int anzahlVokale(String str)
    {
        int vokale = 0;
        for(int i=0; i<str.length(); i++)
        {
            String buchstabe = str.substring(i,i+1);
            if(buchstabe.equals("a") || buchstabe.equals("e") ||
               buchstabe.equals("i") || buchstabe.equals("o") ||
               buchstabe.equals("u")
            )
            {
                vokale = vokale+1;
            }
        }

        return vokale;
    }
}
```

```
// zum Testen:  
public static void main(String[] args)  
{  
    TextObjekt to = new TextObjekt();  
    String x = "vom eise befreit";  
  
    int n = to.anzahlVokale(x);  
  
    System.out.println(n);  
}  
}
```

Interessant ist auch eine Methode, die die “umgedrehte” Zeichenkette berechnet. Zum Beispiel ist *“vom eise befreit”* umgedreht gleich *“tierfeb esie mov”*

Die Idee eines Algorithmus wäre in etwa folgende:

Diese Methode würde in der Klasse **TextObjekt** dem entsprechend wie folgt implementiert werden:

```
public String reverse(String str)
{
    String result = "";
    int len = str.length();
    for(int i=len-1; i>=0; i--)
    {
        String buchstabe = str.substring(i, i+1);
        result = result + buchstabe;
    }

    return result;
}
```

übungsaufgaben

a

Erstellen Sie eine Klasse **TextObjekt**. Implementieren in der Klasse **TextObjekt** eine Methode **bisAufLetzten**, die berechnet, wie der String ohne seinen letzten Buchstaben aussieht.

Wenn str ein String ist, dann gibt der folgende Code:

```
TextObjekt text = new TextObjekt();
String x = text.bisAufLetzten("Winter");
System.out.println(x);
```

also aus: "Winte"

b

Erstellen Sie eine Klasse **TextObjekt**. Implementieren in der Klasse **TextObjekt** eine Methode **ohneVokale**, die berechnet, wie ein String ohne Vokale aussieht (also ohne a,e,i,o,u).

Wenn str ein String ist, dann gibt der folgende Code:

```
TextObjekt text = new TextObjekt();
String x = text.ohneVokale("vom Eise befreit
                          ist der Strom");
System.out.println(x);
```

also aus: "vm s bfrt st dr Strm"

c

Schreiben Sie ebenso eine Methode **allesMitO**, die in ähnlicher Weise alle Vokale durch ein "o" ersetzt. Der obige String würde hier also ergeben:

"vom Oose bofroot ost dor Strom"

(sehr
schwer!!)

d

Schreiben Sie eine Methode **laengstesWort**, die zu einer Zeichenkette x (die als Satz der natürlichen Sprache gesehen wird), das längste Wort von x zurückgibt.

So wäre das längste Wort von

"vom Eise befreit sind Strom und Bäche"

beispielsweise "befreit"

Wichtig: dafür benötigen Sie Wissen über die Methode **indexOf(...)** der String-Klasse!!!

Zuweisung, Identität und Gleichheit

"Box and Pointer" Notation

Um Objekte in Java grafisch darzustellen, wird häufig die "Box and Pointer" Notation verwendet. Dabei wird eine Variable

- durch ein Quadrat dargestellt,
- das mit dem Variablennamen bezeichnet ist,
- und von dem ein Pointer (Pfeil) auf eine Box weist,
- die das Objekt darstellt, das der Variablen zugeordnet ist.
- Ausnahme sind die primitiven Datentypen: hier wird der Wert der Variablen direkt in die Box eingetragen

Beispiel 1:

Die folgenden Variablen würden in Box-and-Pointer Notation wie folgt dargestellt werden:

```
int x = 5;
```

```
double d = 1.7;
```

```
int[] feld = new int[5];  
feld[2] = 17;
```

```
String s = "hallo";
```

Beispiel 2:

Die folgende Klasse wird in unseren nächsten Beispielen verwendet und stückweise

erweitert:

```
public class Person
{
    private int    alter  = 20;
    private String name  = "peter meier";

    public static void main(String[] args)
    {
        Person a = new Person();
    }
}
```

Der hier erreichte Zustand kann in der Box and Pointer Notation wie folgt dargestellt werden:

Erweitern wir unsere Klasse erst einmal wie folgt um die Methoden `print()` und `changeName()` :

```
public void print()
{
    System.out.println(name+" ist "+alter+" jahre alt");
}
```

```
public void changeName(String otherName)
{ name = otherName;
}
```

Nach Aufruf der folgenden main-Methode

```
public static void main(String[] args)
{
    Person a = new Person();
    Person b = new Person();
    b.changeName("petra meier");
    a.print();
    b.print();
}
```

Entsteht der folgende Zustand:

Referenz und Zuweisung

Der Pfeil vom Variablennamen zum zugeordneten Objekt in unserer Darstellung wird **Referenz** genannt.

In Java sind alle Variablen und alle Parameter an ihre zugeordneten Objekte per Referenz gebunden.

Wird einer Variablen ein (neues) Objekt zugewiesen, so bedeutet das also,

- dass die alte Referenz aufgelöst wird
- und die Variable durch eine neue Referenz an das ("neue") Objekt gebunden wird

Beispiel

Erweitern wir unsere Klasse erst einmal durch einen "guten" Konstruktor:

```
public Person(String startName, int startAlter)
{
    name = startName;
    alter = startAlter;
}
```

Verwenden wir die folgende main-Methode

```
Person a = new Person("homer simpson", 36);
a      = new Person("montgomery burns", 85);
```

Dann entsteht der folgende Zustand:

Wir sehen: die Referenz der Variable *a* auf das Objekt wurde aufgelöst und eine neue Referenz auf das neue Objekt aufgebaut.

Garbage Collection

Objekte, auf die keine Referenz mehr existiert, können vom Programm nicht mehr benutzt werden (eben WEIL keine Referenz mehr auf sie besteht).

Diese Objekte können bzw. müssen natürlich aus dem Computerspeicher gelöscht werden, da der Speicher sonst irgendwann "überfüllt" wäre.

In Programmiersprachen wie C, C++ oder Pascal **müssen solche Objekte vom Programmierer im Programmcode explizit gelöscht werden.**

Dazu existiert in C beispielsweise der Befehl `free` oder in C++ der Befehl `delete`.

In Sprachen wie LISP, PROLOG, Scheme oder (eben!) Java ist dies jedoch nicht notwendig.

Hier existiert ein "Hilfsprogramm", das bei Speicherplatzmangel

- alle vorhandenen Objekte durchläuft
- und das Objekt löscht, falls keine Referenz mehr auf das Objekt besteht

Dieses Programmteil wird "garbage collector" ("Müllsammler") genannt, der Aufräumvorgang entsprechend "garbage collection".

Zuweisung Teil 2

Es wurde vorhin erklärt:

Wird einer Variablen ein Objekt zugewiesen, so bedeutet das,

- dass die alte Referenz aufgelöst wird
- und die Variable durch eine neue Referenz an das Objekt gebunden wird.

Das kann auch dazu führen,

- dass auf ein einziges Objekt
- zwei oder mehr verschiedene Referenzen existieren,
- das Objekt also mehreren Variablen "gleichzeitig" zugeordnet ist

Beispiel

Durch die folgende main-Methode

```
Person a = new Person("homer simpson",36);  
Person b = a;  
  
a.print();  
b.changeName("sideshow bob");  
a.print();
```

Entsteht der folgende Zustand:

Ausgabe des Programms ist also

```
homer simpson ist 36 jahre alt  
sideshow bob ist 36 jahre alt
```

Identität und Gleichheit

In Java gibt es zwei Arten, wie zwei Variablen einander "gleich" sein können: Identität und Gleichheit.

Identität

Identität ist die "schärfste" Art von Gleichheit. Identität bedeutet die Frage, ob zwei Variablen dasselbe identische Objekt meinen.

In Java sind zwei Variablen identisch, wenn beide Variablen auf das selbe Objekt referieren.

Auf Identität wird in Java durch den Operator `==` geprüft.

Beispiel

```
public static void main(String[] args)
{
    Person x = new Person("bart",10);
    Person a = x;
    Person b = new Person("bart",10);

    System.out.println("identisch x und a: " + (a==x) );
    System.out.println("identisch x und b: " + (b==x) );
}
```

Zustand im box and pointer diagramm:

Ausgabe des Programms ist also:

identisch x und a: true
identisch x und b: false

Gleichheit

Gleichheit ist "toleranter" als Identität; sie bedeutet die Frage, ob zwei Variablen als "sinnvollerweise gleichwertig" angesehen werden sollten.

Gleichheit muss durch den Programmierer der Klasse "eigenhändig" implementiert werden. Eine Methode zur Überprüfung auf Gleichheit sollte grundsätzlich das folgende Aussehen haben:

```
public boolean equals( Klassenname obj )  
{  
    VergleichsAnweisungen mit Return-Befehl  
}
```

Beispiel

Gleichheit für unsere Beispielklasse sollte wie folgt implementiert werden:

```
public boolean equals(Person anderes)  
{  
    if(name != anderes.name)  
    { return false;  
    }  
    else if(alter != anderes.alter)  
    { return false;  
    }  
    else  
    { return true;  
    }  
}
```

equals

Eine equals-Methode ist in Java für viele Klassen schon standardmässig implementiert, beispielsweise für die String-Klasse.

this

In etwas komplexeren Programmen stehen Objekte oftmals in “Beziehung” zu anderen Objekten der gleichen Klasse.

Beispiel

Personen, die die Möglichkeit zur Heirat haben, können wie folgt modelliert werden:

```
public class Person
{
    private String    nachname;
    private String    vorname;
    private Person    partner;
    private boolean    weiblich;
```

```
public Person(String nn, String vn, boolean w)
{
    nachname = nn;
    vorname  = vn;
    weiblich = w;
    partner  = null;
}
```

```
public void print()
{
    System.out.println(vorname+" "+nachname);
    if(partner != null)
    { System.out.println("verheiratet mit:" +
        partner.vorname);
    }
    System.out.println(".");
}
```

```
public void heirate(Person pers)
{ partner = pers;
}
```

```
public static void main(String[] args)
{
    Person ho = new Person("Simpson", "Homer", false);
    Person ma = new Person("Bouvier", "Marge", true);

    ho.print();
    ma.print();

    System.out.println("-----");
    ho.heirate(ma);

    ho.print();
    ma.print();
}
}
```

Das zugeordnete box and pointer diagram ist:

Ausgabe des Programms ist folglich:

Homer Simpson

Marge Bouvier

Homer Simpson

verheiratet mit: Marge

Marge Bouvier

Das Problem liegt in der Methode `heirate`: eine Person P. zu heiraten bedeutet nicht nur

- das P. der eigene Partner wird, sondern auch, dass
- der Partner von P. ich selber werde

this

das Schlüsselwort **this** in Java hat als Wert die Referenz auf das "aktuelle" Objekt (dessen Methode gerade ausgeführt wird)

Daher sollte wie Methode `heirate` wie folgt reformuliert werden:

```
public void heirate(Person pers)
{
    partner = pers;
    pers.partner = this;
}
```

das box and pointer diagram ist dann:

Vererbung

Beziehungen zwischen Klassen

Betrachten wir einmal die folgenden Klassen:

Blumen, Gänseblümchen, Rosen, Blütenblätter und Marienkäfer

Diese auf den ersten Blick "durcheinander" scheinende Menge läßt anhand verschiedener Beziehungen ordnen:

- die Art "Gänseblümchen" ist eine Art von Blume
- die Art "Rose" ist eine andere Art von Blume
- rote Rosen und gelbe Rosen sind Rosenarten
- ein Blütenblatt ist Bestandteil einer Blume
- Marienkäfer "helfen" Blumen, Schädlinge zu bekämpfen

Klassen aus einem gemeinsamen "Umfeld" existieren also nicht isoliert voneinander, sondern stehen oft Beziehungen zueinander.

Dabei sind stehen folgende Beziehungen im Vordergrund:

Beziehung	Realisierung in Java	Java-Konstrukt
Eigentum ("part")	Instanzvariable	private;
Verwendung ("use")	Methodenaufruf	Punkt-Operator
Vererbung ("is-a")	Vererbung	extends

Von diesen Beziehungen werden wir nun die **Vererbung** näher betrachten.

Das Konzept *Vererbung*

Zwei Klassen stehen in Vererbungs-Beziehung zueinander, **wenn die eine Klasse eine "Unterart" der anderen Klasse ist.**

Man spricht hierbei auch **von der Basisklasse und der abgeleiteten Klasse.**

Dabei **erbt** ("übernimmt") die abgeleitete Klasse automatisch alle Methoden und Instanzvariablen der Basisklasse.

Beispiel

Stellen wir uns wieder eine Klasse "Mensch" vor, die Vorname, Name und Alter besitzt:

```
public class Mensch
{
    private String    vorname;
    private String    nachname;
    private int       alter;

    public Mensch(String vn, String nn, int al)
    {
        vorname = vn;
        nachname = nn;
        alter = al;
    }
}
```

```
public void hatGeburtstag()
{
    alter++;
}
```

```
public void print()
{
    System.out.println(" " + vorname + " " + nachname +
        ", alter:" + alter);
}
```

```
}
```

Grafisch dargestellt (UML):

Stellen wir uns jetzt einen Firmenmitarbeiter vor. Dieser ist

- grundsätzlich ein Mensch
- der noch zusätzlich eine Abteilung und ein Gehalt besitzt
- und dessen Gehalt erhöht werden kann

Grafisch wird dies in UML dargestellt, indem die abgeleitete Klasse mit einem Pfeil auf die Basisklasse zeigt:

Vererbung in Java

extends

das Schlüsselwort `extends` dient in Java dazu, eine Klasse von einer bestimmten **Basisklasse** abzuleiten.

Das Schlüsselwort `extends` wird dabei direkt hinter die abzuleitende Klasse geschrieben.

Direkt hinter `extends` folgt dann der Name der Basisklasse.

Schreibweise:

```
public class Klasse extends Basisklasse
{
    // ... wie bisher ...
}
```

Beispiel:

Die Klasse Mitarbeiter, die von Mensch abgeleitet wird, kann wie folgt definiert werden:

```
import Mensch;
```

```
public class Mitarbeiter extends Mensch
{
    private String  abteilung;
    private double  gehalt;

    // Konstruktor
    public Mitarbeiter(String nn, String vn, String abt)
    {
        nachname = nn;
        vorname   = vn;
        abteilung = abt;
    }

    public void erhoeheGehalt(double betrag)
    { gehalt = gehalt+betrag;
    }

}
```

Der Konstruktor der Klasse `Mitarbeiter` ist in dieser Form jedoch **fehlerhaft**. Das kommt daher, dass

- die Klasse `Mitarbeiter` von `Mensch` abgeleitet ist,
- und daher bei der Konstruktion eines `Mitarbeiters` zuerst versucht wird, einen `Mensch` zu konstruieren,
- dazu ist jedoch ein Konstruktor mit Nachname, Vorname und Alter notwendig!!!!

Für die Basisklasse `Mensch` existiert jedoch ein Konstruktor der Form

`public Mensch(String vn, String nn, int alt)`

In einem solchen Fall muss der Konstruktor der Basisklasse mittels des Schlüsselworts `super` aufgerufen werden:

`super(...)`

das Schlüsselwort `super` dient in Java dazu unter anderem dazu, **den Konstruktor der Basisklasse mit den notwendigen Parametern aufzurufen**.

Ein solcher Aufruf muss im Konstruktor einer abgeleiteten Klasse immer dann stattfinden, **wenn die Basisklasse einen Konstruktor zur Verfügung stellt**.

Der Aufruf mittels `super` **muss dabei die erste Anweisung im Konstruktor der abgeleiteten Klasse sein**.

Schreibweise:

```
super ( notwendigeParameter );
```

Der Konstruktor der Klasse Mitarbeiter wird die folgt geändert und die folgende main-Methode erzeugt:

```
// korrekter Konstruktor
public Mitarbeiter(String vn, String nn,
                    int alt, String abt)
{

    abteilung = abt;
    gehalt    = 3000;
}
```

Implementieren wir nun eine main-Methode wie folgt:

```
public static void main(String[] args)
{
    Mitarbeiter le = new Mitarbeiter("Lenny",
                                    "Lennard", 34, "Produktion");

    le.print();
}
```

Ausgabe des Programms ist dann:

Lenny Lennard, alter: 34

Dies gilt, da die Klasse Mensch ja die Basisklasse der Klasse Mitarbeiter ist.

Folglich wurde die Methode `print()` von dieser Basisklasse übernommen.

Daher werden die zusätzlichen Instanzvariablen `gehalt` und `abteilung` natürlich nicht mit ausgedruckt.

Sollen diese Instanzvariablen mit ausgedruckt werden, so muss die Methode `print()` in der abgeleiteten Klasse `Mitarbeiter` redefiniert werden.

Dazu muss unser Wissen über das Schlüsselwort `super` erweitert werden:

`super.methodName(...)`

das Schlüsselwort `super` bezeichnet in Java die Basisklasse der aktuellen Klasse.

Daher kann es dazu dienen, bewusst eine bestimmte Methode der Basisklasse aufzurufen.

Schreibweise:

```
super.methodName( notwendigeParameter );
```

Dies nutzen wir, indem wir in der Methode `print`

- den `Mitarbeiter` zuerst als normalen `Mensch` ausdrucken lassen
- und dann die zusätzlichen Daten der abgeleiteten Klasse ausdrucken:

```
// in der Klasse **Mitarbeiter** !!!
public void print()
{
    // erst als Mensch ausdrucken:
    super.print();

    // dann die zusätzlichen Daten:
    System.out.println("Abteilung: "+abteilung+
                       " , Gehalt: "+gehalt);
}
```

Dieses Phänomen wird auch Polymorphie genannt:

Polymorphie

Polymorphie wird in Java die Eigenschaft bezeichnet, dass in einer Basisklasse und ihren abgeleiteten Klassen verschiedene Methoden mit dem gleichen Namen existieren können.

Das laufende Programm benutzt beim Aufruf einer polymorph definierten Methode die Methode, die aus der Klasse des angesprochenen Objektes stammt.

Übungsaufgabe

Stellen Sie sich die Waren-Verwaltung eines Supermarktes vor! Implementieren Sie die Java Klasse `Ware`, die - in Bruchteilen - wie folgt aussieht:

```
class Ware
{
    private String    name;
    private long      warennr;
    private double    preis;
    private long      anzahlImLager;

    void Ware(String warenName)
    ...

    void setzePreis(double)
    ...

    void print()
    ...

    void verkaufe(long anzahl)
    ...

    void kaufeEin(long anzahl)
    ...
}
```

Folgendes Programm sollte dann möglich sein:

```
Ware w = new Ware("Holzteile");
w.setzePreis(1.50);
w.kaufeEin(100);
w.verkaufe(4);
w.print();
```

Das Ergebnis sollte sein: Holzteile -- im Lager: 96 -- Preis: 1.5

- a.** Leiten Sie von der Klasse `Ware` die Klasse `Frucht` ab. Diese Klasse soll die weitere Komponente `reifegrad` besitzen (1 für gruen, 2 für reif, 3 für ueberreif) sowie das Datum `verfallsdatum`.

Entsprechende zusätzliche Methoden sollen `setzeReifegrad(...)` und `setzeVerfallsdatum(...)` sein.

Die Methode `print` der Klasse `Frucht` soll natürlich auch `reifegrad` und `verfallsdatum` drucken!

Vererbung und Zuweisung

Objekte einer Basisklasse und einer abgeleiteten Klasse weisen starke "Ähnlichkeiten"

zueinander auf.

Daher stellt sich die Frage, ob **Objekte dieser Klassen gegeneinander zugewiesen werden können.**

Objekte der abgeleiteten Klasse enthalten in Java gegebenenfalls "mehr" Komponenten als in der Basisklasse, da die abgeleitete Klasse ja durch Methoden und Variablen erweitert werden kann.

In Java gilt das Prinzip, dass **Objekte der abgeleiteten Klasse an Variablen der Basisklasse zugewiesen werden können. Die Zuweisung in der anderen Richtung geht nicht !!**

Beispiel:

```
public static void main(String[] args)
{
    Mitarbeiter le = new Mitarbeiter("Lenny", "Leonard",
                                     34,"Produktion");
    le.print();
}
```

```
Mensch me;
me = le;
me.print(); // OK
```

```
Mensch cc = new Mensch("Carl","Carlson",35);
cc.print();
```

```
Mitarbeiter mi;  
mi = cc;          // FEHLER, Übersetzung stoppt  
mi.print();  
}
```

Zum Verständnis dieses Phänomens sollte die u.a. auch die Klasse `Object` bekannt sein:

Object

die Klasse `Object` bezeichnet in Java die Basisklasse aller möglichen Java-Klassen.

Jede Klasse ist in Java entweder direkt oder über mehrere Zwischenklassen von `Object` abgeleitet.

Daher kann jedes beliebige Objekt einer Variablen der Klasse `Object` zugewiesen werden.

Typumwandlung (Type Cast)

Allerdings können dabei zu einer Variablen immer nur die Methoden verwendet werden, die in der Klasse der Variablen vorgesehen sind.

Beispiel:

```
public static void main(String[] args)
{
    Mensch me    = new Mensch("Bart", "Simpson", 10);
    String  st   = "Sommer";

    Object obj;
    obj = st;
    obj = me;

    obj.print();    // FEHLER: print() existiert nicht
                   // in <Object>
}
```

Der Aufruf von `obj.print()` führt daher zu einem Fehler, da die Methode `print()` in der Klasse `Object` nicht definiert ist.

Variablen der Klasse `Object` lassen sich aber dazu verwenden

- beliebige Objekte "zwischenzuspeichern",
- um sie später wieder an eine Variable des "richtigen" Typs zuzuweisen

Genau dazu dient der Cast-Operator in Java:

cast-Operator

der sogenannte **Cast**-Operator dient in Java dazu,

- ein Objekt obj einer bestimmten Klasse
- in ein Objekt einer anderen Klasse K umzuformen,
- wenn dies möglich ist.

Schreibweise:

`(K) obj`

Das obige Programm wird also korrekt durch:

```
Mensch me      = new Mensch("Bart", "Simpson", 10);  
String st      = "Sommer";
```

```
Object obj;  
obj = st;  
obj = me;
```

```
Mensch ba = (Mensch) obj;  // Cast-Operator  
ba.print();
```

public, protected, private

Wir hatten schon recht früh festgestellt, dass in der objektorientierten Programmierung meistens

- Methoden öffentlich genutzt werden können
- während auf Instanzvariablen nur vom Objekt selber zugegriffen werden soll

Dem entsprechen die Zugriffsmodifikatoren

- **public** : auf das Element/Methode kann von "überall" her zugegriffen werden
- **private** : nur das Objekt selber (und Elemente derselben Klasse) können zugreifen

Wenn wir dann folgende Klasse haben:

```
public class Mensch
{
    private String vorname;
    private String nachname;
    private int alter;

    public Mensch(String v, String n, int a)
    {
        vorname = v;
        nachname = n;
        alter = a;
    }

    public void hatGeburtstag()
    { alter++; }

    public void print()
    {
        System.out.println(vorname+" "+nachname+", Alter: "+alter);
    }
}
```

Läuft die folgende main-Methode natürlich korrekt ab:

```
Mensch h = new Mensch("homer", "simpson", 40);
h.hatGeburtstag();
h.print();
```

Dies ist der Fall, da die Methoden `hatGeburtstag()` und `print()` **public** sind und damit öffentlich aufrufbar.

Es würde jedoch der folgende Code nicht funktionieren

```
h.alter = -100;
```

da alter **private** ist und somit nicht öffentlich benutzbar

protected

Definieren wir eine von *Mensch* abgeleitete Klasse *Mitarbeiter*:

```
public class Mitarbeiter extends Mensch
{
    private String abteilung;

    public Mitarbeiter(String v, String n, int a, String abt)
    {
        super(v,n,a);
        abteilung = abt;
    }

    public void print()
    {
        System.out.println(vorname+" "+nachname+", Alter:"+alter);
        System.out.println("Abteilung: "+abteilung);
    }
}
```

Sowie folgende main-Methode:

```
Mensch h = new Mitarbeiter("homer", "simpson", 40, "Sektor 7G");  
h.hatGeburtstag();  
h.print();
```

Dies funktioniert jedoch nicht, da

- die Instanzvariablen `vorname`, `nachname` und `alter` in der Klasse Mensch als `private` definiert sind
- und deswegen in der abgeleiteten Klasse Mitarbeiter nicht auf sie zugegriffen werden kann

Das ist in vielen Fällen jedoch so unerwünscht und unpraktisch. Daher gibt es mit dem Modifikator "protected" eine weitere Regelung für den Zugriff auf Instanzvariable (und Methoden):

- Auf ein als "protected" definiertes Element kann in der Klasse selber (wie bei "private")
- **und** in allen von dieser Klasse abgeleiteten Klassen zugegriffen werden.

Daher müssen diese Instanzvariablen im Beispiel wie folgt definiert werden:

```
public class Mensch  
{  
    protected String vorname;  
    protected String nachname;  
    protected int alter;  
    ...  
}
```

Ausnahmebehandlung (Exceptions)

Methoden dienen dazu, gewisse Operationen auf einem Objekt auszuführen. Nun kann es jedoch passieren,

- dass das Objekt es zum aktuellen Zustand gar nicht erlaubt, diese Methode auszuführen,
- sondern ein "Fehlerzustand" vorliegt

Erweitern wir beispielsweise die Klasse *Mensch* um einen Partner:

```
public class Mensch
{
    protected String vorname;
    protected String nachname;
    protected int    alter;
    private  Mensch partner;

    public Mensch(String v, String n, int a)
    {
        vorname  = v;
        nachname = n;
        alter    = a;
        partner   = null;
    }

    public void hatGeburtstag()
    { alter++;
    }

    public void heirate(Mensch m)
    {
        this.partner = m;
        m.partner    = this;
    }
}
```

```

public void print()
{
    System.out.println(vorname+" "+nachname+", Alter: "+alter);

    if(partner!=null)
    { System.out.println("verheiratet mit: "+partner.vorname);
    }
}
}

```

Dann funktioniert natürlich folgende main-Methode:

```

Mensch h = new Mensch("homer","simpson",28);
Mensch m = new Mensch("marge","bouvier",27);
h.heirate(m);

h.print();

```

Allerdings würde auch folgendes funktionieren:

```

Mensch ho = new Mensch("homer","simpson",28);
Mensch ma = new Mensch("marge","bouvier",27);
Mensch he = new Mensch("helen","lovejoy",29);
ho.heirate(ma);
ho.heirate(he);

```

Was (in vielen Kulturkreisen) jedoch so nicht gehen sollte. Die Methode *heirate* sollte bei einem schon verheirateten Objekt signalisieren, dass hier ein Problem auftritt.

Genau dazu ist die Klasse der Ausnahmen (Exceptions) definiert:

Exception

die Klasse `Exception` und die von ihr abgeleiteten Klassen dienen in Java dazu, das Auftreten eines Fehlers oder eines inkonsistenten Zustandes zu signalisieren.

Das Auftreten eines Fehlers in einer Methode wird von der Methode signalisiert, indem

- ein neues `Exception`-Objekt erzeugt wird
- und dieses Objekt mittels throw signalisiert wird (man spricht auch vom "werfen").
- Mit einem `throw` wird die Methode auch an dieser Stelle beendet (wie bei `return`)

Zudem muss eine Methode, die ein "throw" enthält, dies mittels des Zusatzes "`throws Exception`" auch verkünden.

Beispiel:

Dem entsprechend sollte die Methode *heirate*

- eine `Exception` werfen
- falls einer der Partner schon verheiratet ist

Der Code sieht dann wie folgt aus:

```

public void heirate(Mensch m) throws Exception
{
    if(this.partner!=null || m.partner!=null)
    {
        Exception ex = new Exception();
        throw(ex);
    }

    this.partner = m;
    m.partner    = this;
}

```

try ... catch ...

Das Werfen einer Exception mittels throw dient also dazu, eine Ausnahme zu signalisieren. Doch der Mechanismus hat noch eine zweite Seite:

es ist jemand erforderlich, der

- die geworfene Ausnahme auch "fängt" (registriert)
- und auf sie entsprechend reagiert

Dazu existiert das Konstrukt try... catch... :

try { ...} catch(Exception ex) { ...}

das Konstrukt `try` { tryBlock }
`catch (Exception ex)` { catchBlock }

dient in Java dazu,

- Code einzuklammern, in dem gegebenenfalls eine Exception auftritt (den tryBlock)
- und, falls die Exception auftritt,
- den catchBlock auszuführen

In Java muss

- grundsätzlich jeder Code, in dem eine Exception auftreten könnte
- mittels try...catch... eingeschlossen und behandelt werden
- Dies dient zur Erhöhung der Code-Sicherheit

Beispiel

Dem entsprechend muss die main-Methode von vorhin angesichts der neuen *heirate*-Methode wie folgt definiert werden:

```

// Menschen...
Mensch ho = new Mensch("homer", "simpson", 40);
Mensch ma = new Mensch("marge", "bouvier", 40);
Mensch he = new Mensch("helen", "lovejoy", 42);

// ... heiraten ...
try
{
    System.out.println("heirate marge");
    ho.heirate(ma);

    System.out.println("heirate helen");
    ho.heirate(he);
}
catch(Exception ex)
{
    System.out.println("Heirat unmöglich!!");
}

// ... und reden
ho.print();

```

Ausgabe des Programmes ist dann auch:

```

heirate marge
heirate helen
Heirat unmöglich!!
homer simpson, Alter: 40
verheiratet mit: marge

```

Rekursion